

# Seven Fold Cinema

## Software Requirements Specification

Version 1.0

09/24/2026

Group 7

Marwah Al Tameemi

Zainab Al Tameemi

Joshua Lopez

Prepared for

CS 250- Introduction to Software Systems

Instructor: Gus Hanna, Ph.D.

Fall 2026

## Revision History

| Date       | Description | Author                                             | Comments                                         |
|------------|-------------|----------------------------------------------------|--------------------------------------------------|
| 09/24/2026 | Version 1.0 | Marwah Al Tameemi, Zainab Al Tameemi, Joshua Lopez | Initial team draft and cross-revision submission |
|            |             |                                                    |                                                  |
|            |             |                                                    |                                                  |
|            |             |                                                    |                                                  |

## Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

| Signature | Printed Name      | Title              | Date       |
|-----------|-------------------|--------------------|------------|
|           | Marwah Al Tameemi | Software Eng.      | 09/24/2026 |
|           | Zainab Al Tameemi | Software Eng.      | 09/24/2026 |
|           | Joshua Lopez      | Software Eng.      | 09/24/2026 |
|           | Dr. Gus Hanna     | Instructor, CS 250 | 09/24/2026 |
|           |                   |                    |            |

# Table of Contents

|                                                                  |          |
|------------------------------------------------------------------|----------|
| <b>REVISION HISTORY .....</b>                                    | <b>I</b> |
| <b>DOCUMENT APPROVAL.....</b>                                    | <b>I</b> |
| 1.0 INTRODUCTION .....                                           | 1        |
| 1.1 PURPOSE.....                                                 | 1        |
| 1.2 SCOPE .....                                                  | 1        |
| 1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS .....               | 2        |
| 1.4 REFERENCES.....                                              | 5        |
| 1.5 OVERVIEW.....                                                | 6        |
| <b>2. GENERAL DESCRIPTION.....</b>                               | <b>7</b> |
| 2.1 PRODUCT PERSPECTIVE .....                                    | 7        |
| 2.2 PRODUCT FUNCTIONS .....                                      | 7        |
| 2.3 USER CHARACTERISTICS .....                                   | 7        |
| 2.4 GENERAL CONSTRAINTS .....                                    | 8        |
| 2.5 ASSUMPTIONS AND DEPENDENCIES .....                           | 8        |
| <b>3. SPECIFIC REQUIREMENTS.....</b>                             | <b>8</b> |
| 3.1 EXTERNAL INTERFACE REQUIREMENTS .....                        | 8        |
| 3.1.1 <i>User Interfaces</i> .....                               | 8        |
| 3.1.2 <i>Hardware Interfaces</i> .....                           | 9        |
| 3.1.3 <i>Software Interfaces</i> .....                           | 9        |
| 3.1.4 <i>Communications Interfaces</i> .....                     | 10       |
| 3.2 FUNCTIONAL REQUIREMENTS .....                                | 10       |
| 3.2.1 <i>Movie Catalog and Browsing</i> .....                    | 10       |
| 3.2.2 <i>Seat Reservation and Ticket Purchase</i> .....          | 11       |
| 3.2.3 <i>Private Theater Room Reservation</i> .....              | 12       |
| 3.2.4 <i>Digital Movie Rental</i> .....                          | 13       |
| 3.2.5 <i>Concession Pre-Ordering</i> .....                       | 14       |
| 3.2.6 <i>User Authentication &amp; Account Access</i> .....      | 15       |
| 3.2.7 <i>Administration and Staff Management</i> .....           | 16       |
| 3.3 USE CASES .....                                              | 18       |
| 3.3.1 <i>Use Case #1: Browse Movies and Showtimes</i> .....      | 19       |
| 3.3.2 <i>Use Case #2: Reserve and Purchase Movie Seats</i> ..... | 20       |
| 3.3.3 <i>Use Case #3: Reserve Private Theater Room</i> .....     | 21       |
| 3.3.4 <i>Use Case #4: Rent On-Demand Digital Movie</i> .....     | 22       |
| 3.3.5 <i>Use Case #5: Pre-Order Concessions</i> .....            | 23       |
| 3.3.6 <i>Use Case #6: User Authentication</i> .....              | 25       |
| 3.4 CLASSES / OBJECTS.....                                       | 26       |
| 3.4.1 <i>Customer</i> .....                                      | 26       |
| 3.4.2 <i>Movie</i> .....                                         | 27       |
| 3.4.3 <i>Employee</i> .....                                      | 28       |
| 3.4.4 <i>MovieCatalog</i> .....                                  | 28       |
| 3.4.5 <i>ConcessionOrder</i> .....                               | 29       |
| 3.4.6 <i>Showtime</i> .....                                      | 29       |
| 3.4.7 <i>Payment</i> .....                                       | 30       |
| 3.4.8 <i>DigitalRental</i> .....                                 | 30       |
| 3.4.9 <i>TheaterReservation</i> .....                            | 30       |
| 3.5 NON-FUNCTIONAL REQUIREMENTS .....                            | 31       |
| 3.5.1 <i>Performance</i> .....                                   | 31       |
| 3.5.2 <i>Reliability</i> .....                                   | 31       |
| 3.5.3 <i>Availability</i> .....                                  | 31       |

|                                           |           |
|-------------------------------------------|-----------|
| 3.5.4 Security .....                      | 32        |
| 3.5.5 Maintainability .....               | 32        |
| 3.5.6 Portability .....                   | 32        |
| 3.6 INVERSE REQUIREMENTS .....            | 32        |
| 3.7 DESIGN CONSTRAINTS .....              | 32        |
| 3.8 LOGICAL DATABASE REQUIREMENTS .....   | 32        |
| 3.9 OTHER REQUIREMENTS .....              | 33        |
| <b>4. ANALYSIS MODELS .....</b>           | <b>45</b> |
| 4.1 SEQUENCE DIAGRAMS .....               | 47        |
| 4.3 DATA FLOW DIAGRAMS (DFD) .....        | 47        |
| 4.2 STATE-TRANSITION DIAGRAMS (STD) ..... | 47        |
| <b>5. CHANGE MANAGEMENT PROCESS .....</b> | <b>47</b> |
| <b>A. APPENDICES .....</b>                | <b>47</b> |
| A.1 APPENDIX 1 .....                      | 47        |
| A.2 APPENDIX 2 .....                      | 47        |

## 1.0 Introduction

This Software Requirements Specification (SRS) for the Seven Fold Cinema web platform defines the system's functionality, structural, and quality requirements. It sets clear standards for developers, designers, test engineers, and stakeholders throughout the software development lifecycle.

### 1.1 Purpose

The purpose of this Software Requirements Specification document is to specify the functional and non-functional requirements for the Seven Fold Cinema (Version 1.0). This document provides an overview of the system architecture, external interfaces, security protocols, and workflows needed to supply a digital ticketing, concession, and streaming platform.

#### **Intended Audience:**

**Software Engineers/Developers:** The main task force that will build and ensure system functionality.

**UX/UI Designers:** Need to best understand how user interactions will align with system requirements.

**Project Stakeholders:** Gain an understanding that the end product will match their business needs and expectations.

**Project Managers:** Figure out the best schedule to meet goals and deadlines.

**Quality Assurance (QA) & Test Engineers:** To proactively gain a full understanding of software inner workings to create accurate test cases and validate that the final software works correctly.

### 1.2 Scope

#### **(1) Software Product Identification**

The software product to be produced is the Seven Fold Cinema Web Application, an integrated web-based cinema management and video-on-demand platform.

#### **(2) System Capabilities & Boundaries**

##### **What the System WILL Do**

- **Online Box Office & Seat Reservation:** Offer seat selection in real time through an interactive seat map, temporary seat holds while checking out, and produce digital tickets with QR codes.
- **Private Theater Room Rentals:** Customers can reserve an entire theater auditorium for private screenings, corporate events, or gaming parties. They will also have the option to pick their schedule and choose from a tiered pricing catalog.
- **On-Demand Digital Movie Rentals:** Provide a digital streaming library that allows users to rent digital copies of their choice to watch at home.
- **Pre-Ordered Concession Sales:** Customers can buy pre-ordered concessions such as popcorn, snacks, and drinks when paying for their tickets. An email or

SMS notification will be sent to them when their order is ready for pickup at the counter.

- **Third-Party Authentication & Payments:** Instead of building an in-house solution, we can use Google's and Apple's APIs for easy OAuth 2.0 single sign-on (SSO), along with PCI-DSS to be compliant with credit card and mobile wallet processing.
- **Administrative Management & Analytics:** Offers Role-Based Access Control (RBAC) tailored to the different roles of theater staff, managers, sales teams, and HR so that they can schedule showtimes, manage inventory, issue refunds, or export any financial reports while keeping all tasks respective to their roles.

#### **What the System WILL Not Do**

- **Physical Point of Sale (POS) Hardware Fabrication:** While the system offers software APIs to enable integration with POS systems, it does not provide custom hardware such as physical barcode scanner guns, physical metal turnstiles, credit card terminal pads, or thermal receipt printing hardware.
- **Offline Native Mobile Apps (v1.0):** Version 1.0 is purely a web application designed for use in mobile or desktop browsers; it will not include a native application for iOS or Android in this release.

#### **(3) Application Objectives & Benefits**

- **Increase Ticket Conversion Rate:** Cut checkout time to under 90 seconds by using a three-step navigation process and third-party SSO.
- **Expand Revenue Channels:** Increase revenue incrementally beyond standard ticket sales by implementing private auditorium sessions and home digital rentals.
- **Accelerate On-Site Operations:** Physical queues at the box office can be eliminated by offering 100% pre-ordered online concessions and by using QR code ticket scanning at the entrance to the theater.
- **Enhance Customer Conversion** (via Strategic UI/UX): Use high-contrast, event-driven features and apply colors that are psychologically designed to stimulate appetite; for example, red or orange accents on food menus can increase concession sales.

### **1.3 Definitions, Acronyms, and Abbreviations**

**AES-256 (Advanced Encryption Standard with a 256-bit key):** A key encryption algorithm that uses a 256-bit key size to protect sensitive data and prevent unauthorized access.

**API (application programming interface):** A set of rules that let different software programs talk to each other and share data. Each application with an API will have different rules and tools.

**AWS (Amazon Web Services):** A popular cloud platform that provides on-demand computing power, storage, databases, and many other services online. Companies and developers will choose this method so they do not have to maintain and buy their own data centers or servers;

they can pay only for what they use as they go. It is meant to provide a scalable solution for projects.

**Bandwidth:** The maximum volume of data that can be transmitted through a network connection within a specified time interval, usually calculated in megabits per second (Mbps) or gigabits per second (Gbps).

**Box Office Splits:** These theatrical agreements usually involve paying a percentage of ticket sales (the film rental fee) rather than a flat one-time PPR fee, ensuring the studio and creators are compensated for public exhibitions.

**Brute Force Attack:** It's a hacking method that uses trial and error to brute force passwords, login credentials, and encryption keys.

**CCPA (California Consumer Privacy Act):** A state statute placed to keep user privacy rights and consumer protection for the residents of California. It gives users the right to know how their data is used, where it is used, and a way to opt out of their information being shared.

**Cookies:** A small text file that a website stores on your device through your web browser to either manage your session, personalize the website, or track your browsing habits to analyze site traffic or show relevant advertisements.

**CSS (Cascading Style Sheets):** A stylesheet language used to control how an HTML page looks, including its colors, fonts, spacing, and how it appears on different devices. This is referred to as the skin and outside appearance of a website.

**Domain Name:** A web address like "www.example.com" that gets turned into an IP address to send internet traffic to a server. It is designed to be human-readable.

**GCP (Google Cloud Platform):** A public cloud service from Google that lets people and businesses use tools and servers online. With GCP, you can build and run applications without buying your own computers.

**GDPR (General Data Protection Regulation):** A European privacy law that protects user information and personal data and gives users total control over their own data.

**Git:** A free, open-sourced distributed version control system designed to track changes in source code and files over time.

**Hosting:** A service that provides the servers and storage needed so people can access a website and its services online.

**HTML (HyperText Markup Language):** This is the main language used to organize and show content on the web, such as headings, paragraphs, images, and forms. Think of it as the bones of a website.

**HTML5:** The fifth and current version of HTML (HyperText Markup Language), and is the standard for organizing and showing content on the web.

**IP Address (Internet Protocol):** A unique numerical label assigned to every device to connect to a computer network.

**JavaScript:** This is a programming language used to make web pages interactive and dynamic. It also helps build server-side applications, like those made with Node.js. Such languages act as the muscles of a website, as they give it function.

**JSON (JavaScript Object Notation):** A text-based format used to store and transport data. It helps create clean and easily readable lists of key-value pairs for HTML and JavaScript to carry data back and forth between them.

**Licensing:** The process of getting or giving official permission to do, use, or produce something.

**Mean Time Between Failures (MTBF):** The average working time a repairable system/machine runs between unexpected breakdowns. It measures how dependable and stable the technology is during everyday operations.

**OAuth 2.0:** A system that lets our web application safely request limited access to the user's account, like Google or Apple, without needing the user's personal password.

**OIDC (OpenID Connect):** A layer built on top of the OAuth 2.0 framework. While OAuth 2.0 focuses on authorization, such as granting permission to access data, OpenID Connect adds authentication, which means verifying who the user is. This lets our website safely confirm a user's identity.

**PCI-DSS (Payment Card Industry Data Security):** A set of required security rules that protect cardholder data and help prevent credit card fraud during payment processing.

**POS (Point of Sale):** This refers to the software and hardware businesses use to mainly handle sales, track inventory, and accept payments for both in-store and online.

**PostgreSQL:** A free database system that can handle complex searches and keeps data safe and organized. Intended for projects just starting out.

**Public Performance Rights (PPR):** Public performance rights to legally screen films to an audience.

**QR Code:** A barcode that stores data inside a grid, designed for quick response and access.

**RESTful API:** A way of designing APIs that uses standard web methods like GET, POST, PUT, and DELETE to let clients and web services exchange information.

**Role-Based Access Control (RBAC):** A system with different roles and permissions for different users to allow for extra security and separation of data. It helps companies follow privacy rules by restricting system access based on a person's role in an organization.

**SEO (Search Engine Optimization):** To rank high in search engine results by optimizing web pages and their content to align with their overall brand for greater visibility.

**SQL (Structured Query Language):** A language used to query and manage relational databases.

**SRS (Software Requirements Specification):** A document that explains exactly what a software system should do. It mostly covers what the system can do, how it should perform, and how it connects with other systems.

**SSL Certificate:** A digital file added to a web server to prove a website's identity and enable encrypted connections (HTTPS) between the browser and server, providing a secure connection.

**SSO (Single Sign-On):** The authentication method that lets users log in to their account (like Google or Apple).

**Terms of Use/ Terms of Service (ToS):** A foundational legal agreement between your website and its users. It outlines the rules for using the service and includes critical clauses like Limitation of Liability, Class Action Waiver, and Governing Laws.

**Theatrical Licensing:** Traditional movie theaters do not buy a standard consumer DVD or streaming file and add a simple PPR. Instead, they enter into formal booking and distribution agreements with film studios or booking agencies.

**TLS (Transport Layer Security):** A cryptographic protocol that provides secure, encrypted communications and data integrity over a computer network.

**UI (User Interface):** The look and layout of a website or app, including buttons, fonts, colors, and other design elements users see and use.

**URL (Uniform Resource Locator):** A unique address used to find a specific webpage on the internet.

**UX (User Experience):** The overall experience a user has when using a system, including how easy it is to use, how logical it feels, and how smoothly it works.

## 1.4 References

- [1] Modern Requirements, "Software Requirements Specification (SRS): A Comprehensive Guide," *Modern Requirements Blog*, 2023. [Online]. Available: <https://www.modernrequirements.com/blogs/requirements-specification/>. [Accessed: Sep. 17, 2026].
- [2] F. Faijati, "How UI/UX Design Directly Affects Conversion and Revenue," *Medium*, Nov. 23, 2022. [Online]. Available: <https://medium.com/@faridafaijati/how-ui-ux-design-directly-affects-conversion-and-revenue-2b770c9ca50b>. [Accessed: Sep. 17, 2026].
- [3] IBM, "What is Role-Based Access Control (RBAC)?," *IBM Think Topics*, 2024. [Online]. Available: <https://www.ibm.com/think/topics/rbac>. [Accessed: Sep. 17, 2026].

- [4] National Center for Biotechnology Information, "Understanding Color and Appetite: Psychological Impact of Visual Stimuli," *PubMed Central (PMC)*, Nov. 2020. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC7658407/>. [Accessed: Sep. 17, 2026].
- [5] AWS & Google Cloud Networking Teams, "AWS and Google Cloud Collaborate to Simplify Multicloud Networking," *AWS Networking & Content Delivery Blog*, 2023. [Online]. Available: <https://aws.amazon.com/blogs/networking-and-content-delivery/aws-and-google-cloud-collaborate-to-simplify-multicloud-networking/>. [Accessed: Sep. 17, 2026].
- [6] I. Redible, "HTML5 Feature Detection Guide," *DEV Community*, 2021. [Online]. Available: <https://dev.to/igcredible/html5-feature-detection-5d7a>. [Accessed: Sep. 17, 2026].
- [7] FusionAuth, "How Single Sign-On (SSO) Works," *FusionAuth Articles*, 2023. [Online]. Available: <https://fusionauth.io/articles/authentication/how-sso-works>. [Accessed: Sep. 17, 2026].
- [8] IBM, "What is Mean Time Between Failures (MTBF)?," *IBM Think Topics*, 2024. [Online]. Available: <https://www.ibm.com/think/topics/mtbf>. [Accessed: Sep. 17, 2026].
- [9] GDC Technology Limited, "Digital Cinema Solutions and Media Servers," *GDC Tech*, 2026. [Online]. Available: <https://www.gdc-tech.com/>. [Accessed: Sep. 17, 2026].
- [10] Fortinet, "What is a Brute Force Attack?," *Fortinet Cyber Glossary*, 2024. [Online]. Available: <https://www.fortinet.com/resources/cyberglossary/brute-force-attack>. [Accessed: Sep. 17, 2026].
- [11] Brave Software, "What Are Terms of Service (ToS)?," *Brave Glossary*, 2023. [Online]. Available: <https://brave.com/glossary/terms-of-service/>. [Accessed: Sep. 17, 2026].
- [12] S. Chacon and B. Straub, *Pro Git*, 2nd ed., "Getting Started - What is Git?," *Git SCM Documentation*, 2014. [Online]. Available: <https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F>. [Accessed: Sep. 17, 2026].

## 1.5 Overview

The remainder of this document is organized as follows. Section 2, General Description, presents the Seven Fold Cinema products at a high level: how they relate to existing box-office and streaming products, the major functions it performs, the characteristics of its intended users, and the general constraints and assumptions that shape the requirements. Section 3, Specific Requirements, contains the detailed functional and non-functional requirements the development team must satisfy, including external interface requirements, individual feature specifications, use cases, and quality attributes such as performance, reliability, and security. Sections 4 and 5, along with the Appendices in upcoming project deliverables, will explain the system's analysis models, the change-management process, and other supporting documents.

## 2. General Description

This section defines the context, user roles, functionality, and general constraints governing Seven Fold Cinema. It provides background information on how the web platform will compare similar products in the market and how it will operate as a modern cinema management system.

### 2.1 Product Perspective

Seven Fold Cinema is a new, self-contained web application that is not a part of a larger pre-existing system. It replaces the theater's current processing system and functions similarly to major commercial platforms such as AMC Theatres' application. However, unlike those platforms, Seven Fold Cinema also lets customers rent an entire theater room for private screenings and rent digital copies of films for at-home viewing. It combines traditional box office ticketing with a small-scale video-on-demand service. The system depends on external services for authentication (Google Auth/Apple OAuth), payment processing, and web hosting, but owns and manages its own database of showtimes, seats, orders, rentals, and users.

### 2.2 Product Functions

This subsection of the SRS provides a summary of the following functions that the software will perform:

- Browse movies, showtimes, and theater locations.
- Reserve and purchase seats for a specific showtime.
- Rent a theater room for a private screening/event.
- Rent a digital copy of a film for at-home viewing.
- Pre-order food and beverage concessions for pickup at showtime.
- Authenticate via email/password or third-party SSO (Google/Apple) and pay securely through an integrated payment gateway.
- Provide administrators with tools to manage showtimes, pricing, inventory, and to view sales analytics
- Provide Customer Service access to a user's login to help troubleshoot and perform actions like processing refunds and updating an account record that users cannot access.

### 2.3 User Characteristics

- Customers: General members of the public across a wide range of ages and technical skill levels. They expect a simple, mobile-friendly browsing and checkout flow and may use the system occasionally
- Box Office / Concession Staff: Theater employees who use an internal/admin view to look up orders, scan QR-code tickets, and fulfill concession pickups. Moderate technical skills and system training.
- System Administrators / Managers: Manage showtimes, pricing, licensing records, and view sales and attendance analytics. Requires elevated access via Role-Based Access Control (RBAC).
- Studio / Content Coordinators (limited use): Supply film metadata and licensing information used to populate the catalog if necessary.

## 2.4 General Constraints

- All payment processing must comply with PCI-DSS, so no raw cardholder data may be stored on Seven Fold Cinema's own servers.
- The system must comply with applicable data-privacy regulations, including GDPR (for EU users) and CCPA (for California residents).
- Any film made available for public screening, private rental, or digital rental must have a valid theatrical/digital licensing or Public Performance Rights (PPR) agreement before it can appear in the catalog.
- The User Interface (UI) and all interactive features, like dynamic seat maps, must render and function on the latest versions of popular web browsers such as Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge without any third-party applications.
- Must run on third-party cloud hosting such as AWS or GCP and will not rely on specialized physical hardware, as this would incur higher upfront costs and be less scalable.
- All web traffic shall be served to end users over public networks and shall be served over HTTPS using a valid, CA-issued SSL/TLS certificate.

## 2.5 Assumptions and Dependencies

- The system must rely on Stripe for credit card payments. Payment processing is reliant on Stripe being operational.
- Authentication and SSO must support Google and Apple OAuth APIs for single sign-on functionality. If any of these external services are down, SSO will be temporarily unavailable.
- System infrastructure relies on an active domain registration with a properly configured DNS for successful routing, a valid SSL/TLS certificate for security, and a PostgreSQL database on a cloud hosting environment to store, manage, and retrieve structured data.
- It is assumed that customers have an active internet connection and that their devices can run modern JavaScript to interact with interactive elements such as a real-time seating chart.
- Customer devices must also support HTML5, as they provide video decoding and the web standards required to display the interactive SVG seat maps and stream on-demand video rentals.

## 3. Specific Requirements

The following section details all the functional and non-functional software requirements needed for the design, development, and testing of the Seven Fold Cinema web platform; it defines the system's external interfaces (covering users, software, hardware, and communication), specifies the detailed functional capabilities and use cases, describes the data models, and sets up attributes such as performance, availability, and security compliance.

### 3.1 External Interface Requirements

#### 3.1.1 User Interfaces

The system will provide:

- A movie catalog interface that will show all the currently showing movies in the theater, as well as an all-time catalog for digital rentals.
- An interactive seating interface that will allow customers to view available and unavailable seats for a specific movie and its showtime.
- A checkout interface that will allow customers to review their selected tickets, theater room, digital rental, the price, and the time if selected.
- A different interactive theater room reservation interface that allows the user to select a date, time, theater room, and the number of guests.
- An interactive user account interface that will hold customer purchases, reservations, digital rentals, and account information.
- A login and registration interface for users and employees/customer service workers.

### **3.1.2 Hardware Interfaces**

The system will:

- Be accessible through desktop computers, laptops, tablets, and phones, all through an internet connection.
- Support devices with a compatible web browser for accessing the website, such as Chrome or Firefox.
- Allow customers to watch their digital movie rentals on said compatible web browsers through the website.

### **3.1.3 Software Interfaces**

The system will:

- Have an interface with a PostgreSQL database that stores and retrieves information related to the customers, movies, showtimes, theater rooms, reservations, seats, purchases, concessions, private theater rentals, and user account information.
- Interface with Stripe or a similar approved payment-processing service to securely process ticket purchases, concession orders, private theater rentals, and digital movie rentals.
- Interface with Google and Apple authentication services using OAuth 2.0 and OpenID Connect (OIDC) to allow users to sign in using third-party accounts.
- Provide an internal authentication system for users who choose to register using an email address and password combination instead of a third-party Single Sign-On service.
- An interface with an email and/or SMS notification service to send users purchase confirmations, digital tickets and QR codes, concession pickup notifications, private theater reservation confirmations, account-related notifications, and other necessary system messages.
- Interface with a digital video streaming service to provide customers with access to their digital rentals.
- An administrative software interface that allows authorized users, such as employees and managers, to manage showtimes, theater rooms, pricing, inventory, reservations, refunds, and other information according to their Role-Based Access Control (RBAC) permissions.

### **3.1.4 Communications Interfaces**

- HTTPS for communication between users' web browsers and the Seven Fold Cinema web application.
- Use TLS encryption for network communications involving authentication information, account information, payment transactions, reservations, and other sensitive data.
- Communicate with external services, including Stripe, Google, Apple, and notification providers, all through their supported web APIs.
- The client's web interface and the server will exchange information using standard web requests and structured data formats such as JSON.
- Maintain secure communication between the application server and the PostgreSQL database so that information such as reservations, customer accounts, orders, showtimes, and inventory can be exchanged without unauthorized access.
- Establish and maintain communication with the payment processor during checkout so that the system can determine whether a transaction was approved or declined if a purchase or reservation fails.

## **3.2 Functional Requirements**

This section defines the critical functional requirements of the Seven Fold Cinema web application. Each requirement below describes the information provided to the system, the system's processing, expected behavior, and actions taken when errors occur.

### **3.2.1 Movie Catalog and Browsing**

#### **3.2.1.1 Introduction**

The system will allow customers to browse current movies, view movie information, and locate available showtimes. The movie catalog includes movies currently playing in theaters as well as digital movies that are available for rental.

#### **3.2.1.2 Inputs**

- Movie selected by the customer
- Theater location
- Selected date
- Selected showtime
- Selection of digital rentals

#### **3.2.1.3 Processing**

- Retrieve movie information from the movie catalog.
- Retrieve showtimes associated with each movie.
- Display only showtimes and movie options currently active in the system.
- Allow customers to select a movie and direct them to the appropriate payment screen.

#### **3.2.1.4 Outputs**

- Movie title
- Movie description
- Available dates and showtimes
- Movie format or viewing option

- Ticket or rental pricing
- Available digital rental options

### **3.2.1.5 Error Handling**

The system will display an error message and prevent the customer from continuing if a movie or showtime cannot be retrieved.

If a requested movie or showtime becomes unavailable, the system will notify the customer and return the customer to available movies or showtime options.

## **3.2.2 Seat Reservation and Ticket Purchase**

### **3.2.2.1 Introduction**

The system will enable customers to choose available seats for a particular movie showtime and buy tickets; they can select up to five seats for one showing.

### **3.2.2.2 Inputs**

- Selected movie
- Selected theater location
- Selected date and showtime
- Selected auditorium
- Up to five seats can be selected
- Customer authentication information, if required
- Customer payment information
- Optional concession selections

### **3.2.2.3 Processing**

- Retrieve the current seating arrangement for the chosen showtime.
- Find which seats are open and which are taken.
- Prevent customers from choosing a seat that is already booked.
- Customers may choose up to five available seats.
- Place a temporary hold on the selected seats for ten minutes while the customer completes checkout.
- Calculate the total ticket cost.
- Let the customer add any concessions before they finish checkout.
- When authentication is required, authenticate the user.
- Send the payment details to the payment processor.
- Confirm the transaction before permanently reserving the selected seats.
- After a successful payment, update the seat status in the database.
- Generate a digital ticket with a QR code.

### **3.2.2.4 Outputs**

- Purchase confirmation.
- Information regarding selected movies and showtimes
- Selected seat number.
- Total amount paid
- Digital ticket or tickets

- A QR code for entering the theater
- An email and/or SMS notification when contact details are available

#### **3.2.2.5 Error Handling**

If the ten-minute seat hold expires before checkout is completed, the system will:

- Release the selected seats.
- Remove the expired reservation from the customer's checkout session.
- Inform the customer that their reservation has expired.

If payment is declined, the system shall:

- Inform the customer that the payment was unsuccessful.
- Let the customer give us another payment method that is supported.
- Not permanently book the seats until a successful transaction occurs.

If a seat is chosen but becomes unavailable before the system establishes a temporary hold, the system will inform the customer and ask them to choose a different seat.

### **3.2.3 Private Theater Room Reservation**

#### **3.2.3.1 Introduction**

The system will allow registered customers to book an entire theater room for a private screening, an event, or for any other approved private use.

#### **3.2.3.2 Inputs**

- Customer account information
- Reservation date
- Starting time
- Desired reservation duration
- Event type
- Number of guests
- Selected theater room or package
- Agreement to Terms of Service
- Payment information

#### **3.2.3.3 Processing**

- Verify the customer is logged into a valid account.
- Check the theater schedule to see which rooms are available.
- Take the requested time and compare it with the current movie showtimes and private reservations.
- Show the available theater rooms and the pricing options.
- Calculate the reservation price based on the room chosen, the time, the duration, and the package.
- The customer must agree to the Terms of Service before they can make a payment.
- Handle the required deposit or full payment.
- After payment, book the chosen theater room and time slot.
- Log the reservation into the database.

#### **3.2.3.4 Outputs**

- Theater room information
- Reservation date and time
- Reservation duration
- Guest count
- Price and details regarding payment
- Confirmation number
- Reservation confirmation by email and/or SMS

### **3.2.3.5 Error Handling**

If the requested theater room or time slot is unavailable, the system will:

- Inform the customer that their requested reservation cannot be carried out.
- Show alternative rooms or time slots that are available.

If payment fails, the system must not confirm a reservation and inform the customer that another payment method is needed.

## **3.2.4 Digital Movie Rental**

### **3.2.4.1 Introduction**

Registered customers can rent movies for at-home streaming via the Seven Fold Cinema web application.

### **3.2.4.2 Inputs**

- Customer account
- Selected digital movie
- Customer's confirmation of the rental price
- Payment information
- Customer's request to start video playback

### **3.2.4.3 Processing**

- Verify the customer is logged into an active account.
- Show the rental price before checking out.
- Make sure the rented movie is available in the customer's account.
- Send the rental payment through the approved payment processor.
- After the payment is made, link the rental to the customer's account.
- Verify that the digital distribution license is valid for the chosen movie.
- Manage the 48-hour rental period from the moment the customer begins playback.
- Each time the customer tries to access the movie, verify the rental period has not expired.
- Play back the video using a compatible HTML5 video player.

### **3.2.4.4 Outputs**

- Rental confirmation
- Movie title
- Rental price
- Rental status
- Time left to rent after you start watching

- Digital access to the movie through the user's account
- Video playback controls

#### **3.2.4.5 Error Handling**

- If the payment fails, the system must refuse the customer access to the movie and will not charge them or activate a rental.
- If the movie's license is revoked for digital rental, the system will stop the transaction and inform the customer that the film is no longer available during checkout.
- If the customer's browser or device cannot support HTML5 playback features, the system will show an incompatibility error.
- Once the 48-hour rental period has expired, the system must stop any further playback and must let the customer know their rental has expired.

### **3.2.5 Concession Pre-Ordering**

#### **3.2.5.1 Introduction**

Customers will be able to pre-order concession items that are available before they reach the theater. Customers can order concessions either during ticket purchase or through the concession ordering section on the website.

#### **3.2.5.2 Inputs**

- Selected concession items
- Quantity of each item
- Available item customizations
- If applicable, the associated movie ticket or showtime
- Customer or guest contact details
- Payment information

#### **3.2.5.3 Processing**

- Get the current concession menu.
- Show what is in stock and how much each item costs.
- Retrieve current inventory information.
- Let the customer choose items and the quantity of each.
- Work out the total amount of the concession order.
- Check that the item is available before completing the checkout.
- Make the payment using the approved payment processor.
- Issue a concession order once payment has been successfully made.
- Keep the order as pending until the theater staff has prepared it.
- Allow authorized theater staff to change order status.
- Complete the order when it is ready for collection.

#### **3.2.5.4 Outputs**

- Stock level of the selected concessions
- Order total
- Payment confirmation

- Order number
- A QR code or some other identifier of the order
- Order status
- An email and/or a text notification when an order is ready

### **3.2.5.5 Error Handling**

If an item becomes unavailable before checkout is completed, the system shall:

- Notify the customer.
- Remove or stop the purchase of the unavailable item.
- Recalculate the order total.

If payment fails, the system will not generate a complete concession order and will allow the customer to try another supported payment method.

## **3.2.6 User Authentication & Account Access**

### **3.2.6.1 Introduction**

The system will authenticate customers and authorize employees before they can access functions specific to their role.

### **3.2.6.2 Inputs**

- Email address and password used for user authentication
- Third-party authentication is available via Google or Apple
- Authentication tokens if the user is already logged in
- If applicable, account recovery information

### **3.2.6.3 Processing**

- Check the provided email and password hash against the securely stored account information.
- Allow Google and Apple as third-party authentication methods via OAuth 2.0 and OpenID Connect.
- Check the authentication information before establishing an active user session.
- Determine the role assigned to the user.
- Assign Role-Based Access Control permissions to decide which features the authenticated user can access.
- Set up a secure session after successful authentication.
- Take the user to the correct customer or employee interface.

### **3.2.6.4 Outputs**

- User's account access.
- Access to features allowed to the user's assigned role.
- The correct interface for the customer, staff, manager, or administrative personnel.

### **3.2.6.5 Error Handling**

- If the entered email address or password is invalid, the system will:

- Deny access.
  - Display an authentication error without revealing if the queried account exists.
  - Give the user the option to retry or use the account recovery procedure.
- When Google or Apple's authentication service is unavailable, the system will inform the user and allow them to try another supported login method.
- For security reasons, the system will log any repeated or failed login attempts.

### **3.2.7 Administration and Staff Management**

#### **3.2.7.1 Introduction**

The system will provide authorized theater employees and managers with the administrative capabilities needed to manage theater operations. The available functions depend on the permissions assigned to each employee via Role-Based Access Control.

#### **3.2.7.2 Inputs**

- Movie information
- Showtime schedules
- Theater room information
- Ticket pricing
- Concession pricing
- Concession inventory levels
- Customer order information
- Refund requests
- Private theater reservation information
- Licensing information
- Updates to customer records that are authorized by employees

#### **3.2.7.3 Processing**

- Before displaying administrative functions to an employee, verify that their identity and assigned role are valid.
- Keep employees' access separate from functions not assigned to them.
- Allow authorized users to create, modify, or delete movie showtimes.
- Let authorized users update pricing information.
- Let authorized users update the concession inventory.
- Let staff find customer orders and reservations.
- Allow authorized employees to handle refunds.
- Let concession staff update the status of concession orders.
- Allow managers to see sales and attendance information.
- Log administrative changes when appropriate for accountability and system tracking.

#### **3.2.7.4 Outputs**

- Updated showtime information
- Latest movie and pricing information
- Current concession inventory stock

- Information regarding customer orders and reservations
- Refund confirmation
- Current theater room reservation information
- Sales and attendance information
- Licensing information
- Confirmation when an administrative change has occurred

#### **3.2.7.5 Error Handling**

- The system will deny access and display a message if an employee tries to access a privileged function for which they are not authorized.
- If an administrative change cannot be made, the system will preserve the existing settings and notify the employee that the operation failed.
- If an attempt to update system information conflicts with existing information, such as creating two showtimes or making theater reservations for the same theater room at the same time, the system will reject this change and notify the employee.

### 3.3 Use Cases

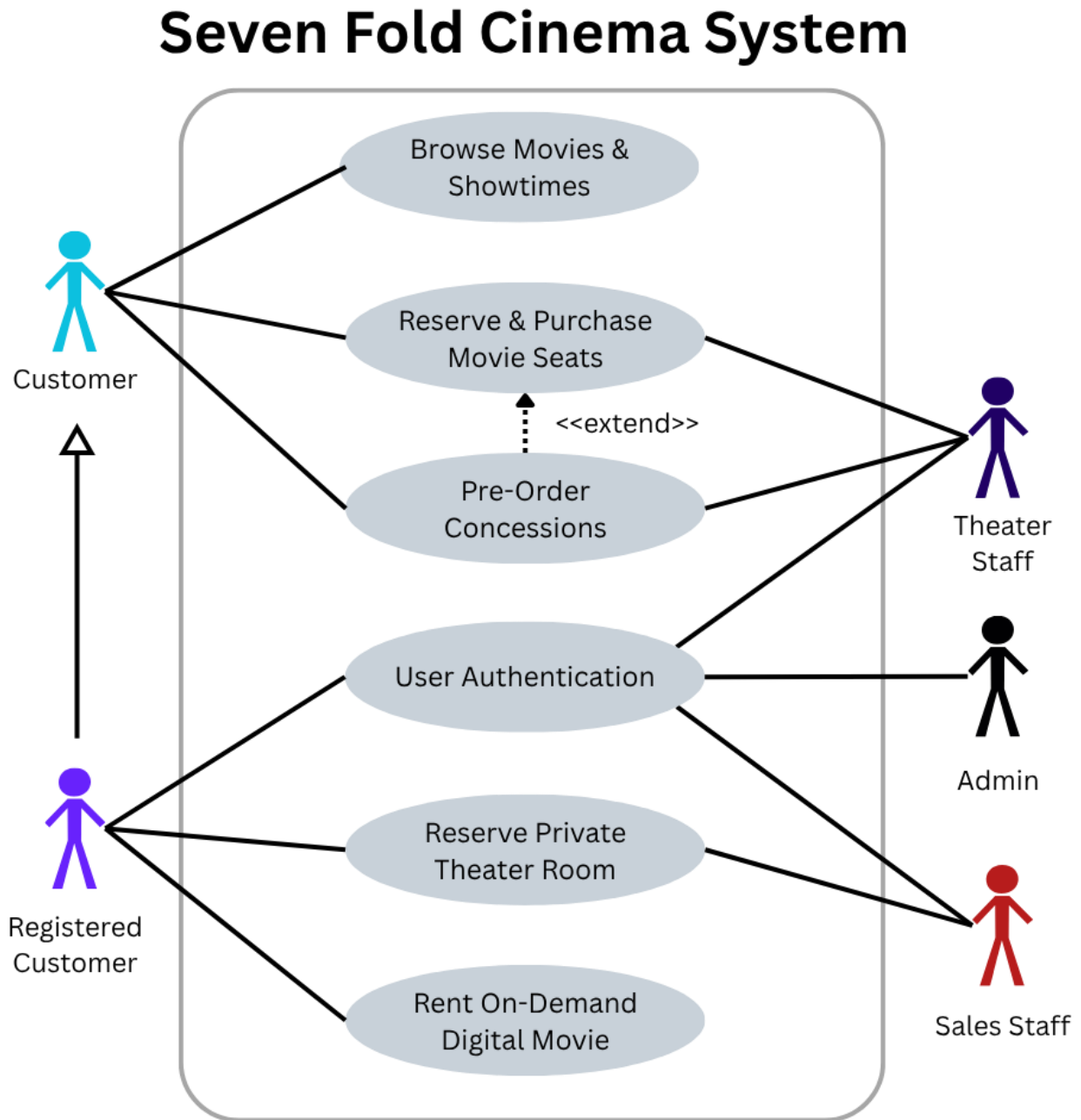


Figure 1. Seven Fold Cinema System Use Case Diagram

## 3.3.1 Use Case #1: Browse Movies and Showtimes

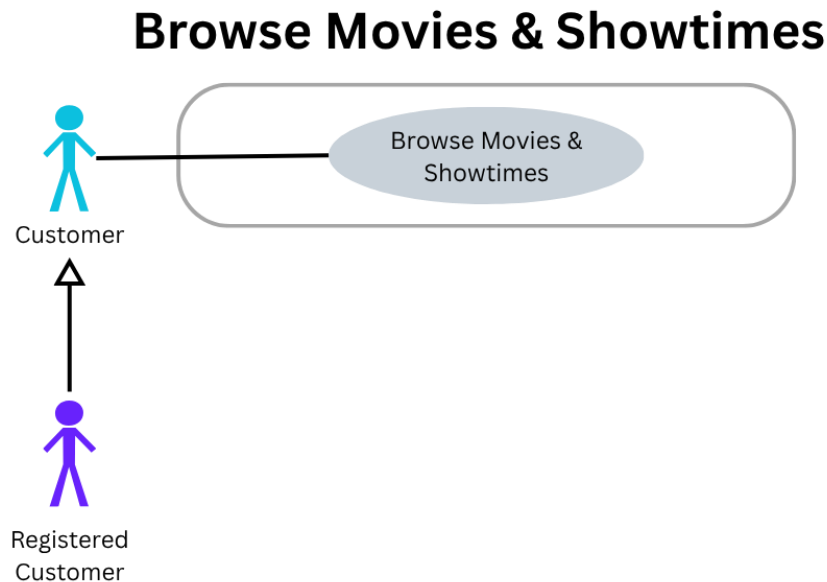


Figure 2. Browse Movies &amp; Showtimes Use Case Diagram

**Goal in Context:** Allows customers to browse currently available movies, view movie information, and locate available showtimes.

**Preconditions:**

- (1) The movie catalog must be available
- (2) Active movie and showtime information must exist in the system.

**Success End Condition:** Customers successfully view movie information and available showtimes and may select a movie/showtime to continue to ticket purchasing.

**Failure End Condition:** Movie or showtime information cannot be retrieved, and the system displays an error message.

**Primary Actor:** Customer

**Trigger:** Customer selects "Browse Movies" or accesses the movie catalog

**Description:**

| Step | Action                                                                  |
|------|-------------------------------------------------------------------------|
| 1    | Customer opens the movie catalog.                                       |
| 2    | System displays currently available movies.                             |
| 3    | Customer selects a movie.                                               |
| 4    | System displays movie information and available showtimes.              |
| 5    | Customer selects a theater location, date, and showtime.                |
| 6    | System displays the selected showtime information.                      |
| 7    | Customer may continue to the Reserve and Purchase Movie Seats use case. |

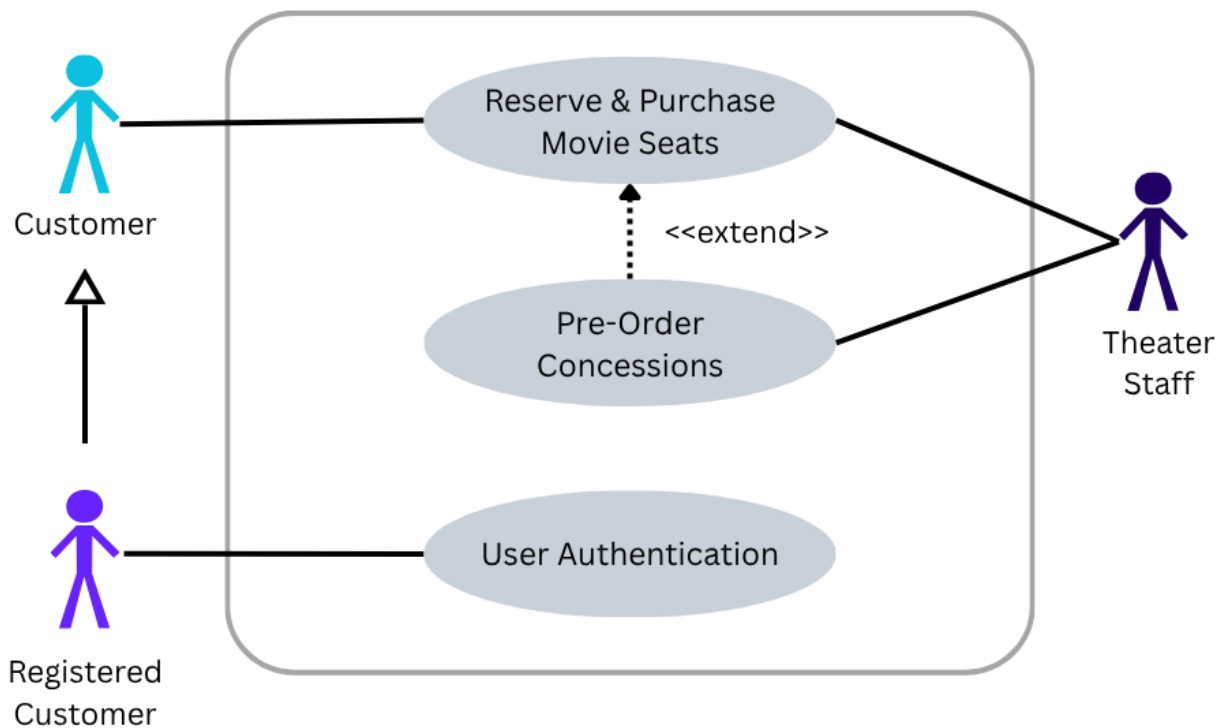
**Extensions or Variations:**

| Step | Branching Action |
|------|------------------|
|------|------------------|

|                             |                                                                                                                                                          |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>No Movies Available</b>  | If no movies are available, the system displays a message indicating that no active listings are available.                                              |
| <b>Showtime Unavailable</b> | If a selected showtime becomes unavailable, the system removes it from the currently available options and asks the customer to select another showtime. |

### 3.3.2 Use Case #2: Reserve and Purchase Movie Seats

## Reserve & Purchase Movie Seats



**Figure 3. Reserve & Purchase Movie Seats Use Case Diagram**

**Goal in Context:** Allows a customer to reserve up to 5 seats and make a purchase on them.

**Preconditions:**

- (1) The selected showtime must have available seats, and the customer must not select more than five seats for the transaction.
- (2) Interactive seat mapping system must be online and connected.

**Success End Condition:** Customers successfully reserve seats, complete payment, and receive a digital ticket with a QR code.

**Failure End Condition:** Reservation is canceled, seats are released back to inventory, and no charge is applied to the customer's account.

**Primary Actor:** Customer

**Secondary Actor:** Theater Staff

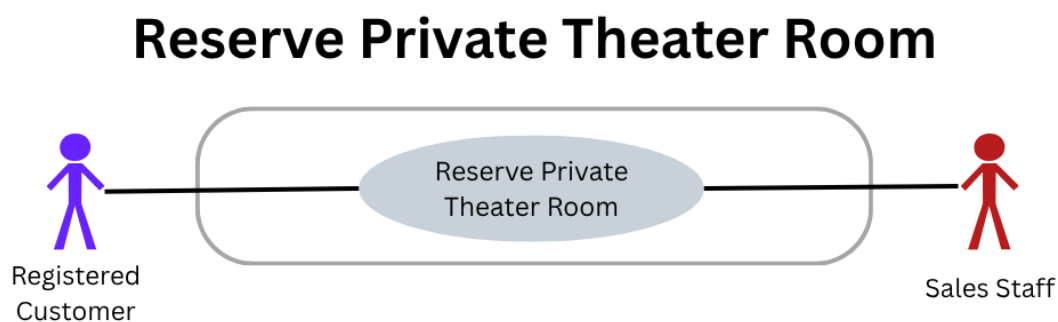
**Trigger:** Customer selects "Reserve Now".

**Description:**

| Step | Action                                                                                                                               |
|------|--------------------------------------------------------------------------------------------------------------------------------------|
| 1    | System displays an interactive seat map, which shows all available seats in real-time.                                               |
| 2    | Customer selects one or up to 5 available seats from the layout.                                                                     |
| 3    | System places a temporary 10-minute hold on the selected seats and starts a checkout timer.                                          |
| 4    | System prompts the user to pre-order any concessions (<<extend>> Pre-Order Concessions).                                             |
| 5    | If the customer signs in to a registered account, the system asks for authentication. If not, the customer can check out as a guest. |
| 6    | Customer selects payment method and submits billing details via Stripe.                                                              |
| 7    | System processes transaction, updates seat status to "Reserved" in PostgreSQL, and generates a ticket.                               |
| 8    | System sends an order confirmation to the customer along with the ticket QR code via email/SMS.                                      |

**Extensions or Variations:**

| Step                            | Branching Action                                                                                                                                                                                    |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Reservation Hold Expires</b> | If the 10-minute hold expires before the payment confirmation, the system will release the selected seats back to inventory, clears the cart, and notifies the customer of the expired reservation. |
| <b>Payment Failure</b>          | If Stripe declines the payment, the system will notify the customer with an error message and prompt them for an alternative payment method.                                                        |
| <b>Staff-Assisted Purchase</b>  | Theater Staff can perform seat selection and ticket checkout on behalf of the customer when assistance is requested.                                                                                |

**3.3.3 Use Case #3: Reserve Private Theater Room****Figure 4. Reserve Private Theater Room Use Case Diagram****Goal in Context:** Allow a customer to reserve a private theater.**Preconditions:**

- (1) Customer must be logged in with a valid user account.

(2) Theater calendar schedule and pricing tiers must be updated by management.

**Success End Condition:** Customer has a selected theater room locked in for private use on a specific date and time, given that a deposit or full payment has been placed and logged.

**Failure End Condition:** Theater room remains open during standard showtimes and no booking is created.

**Primary Actor:** Registered Customer

**Secondary Actor:** Sales Staff

**Trigger:** Registered customer selects "Reserve Now" in "Private Theater Rental" Portal.

**Description:**

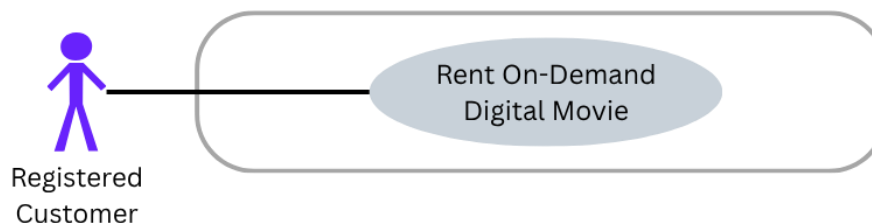
| Step | Action                                                                                                                                                               |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Customer selects their target date, time duration slot, event type, and guest count.                                                                                 |
| 2    | System searches for its availability database and displays available rooms with pricing options.                                                                     |
| 3    | Customer selects their desired room package.                                                                                                                         |
| 4    | System displays a reservation summary, fee breakdown, and ToS for the venue.                                                                                         |
| 5    | Customer accepts the ToS and inputs their billing details.                                                                                                           |
| 6    | System processes the transaction through the payment gateway, locks the theater room time slot in the database, and pushes a reservation confirmation via email/SMS. |

**Extensions or Variations:**

| Step                         | Branching Action                                                                                                                                                                 |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Time Slot Unavailable</b> | System alerts the customer that the selected time slots and theater room are unavailable for private bookings at the time specified and offers alternative available time slots. |

### 3.3.4 Use Case #4: Rent On-Demand Digital Movie

## Rent On-Demand Digital Movie



**Figure 5. Rent On-Demand Digital Movie Use Case Diagram**

**Goal in Context:** Allow customers to rent on-demand movies.

**Preconditions:**

- (1) Customer is logged into an active account.
- (2) Selected digital title has an active licensing agreement to allow distribution.

**Success End Condition:** Customer is granted a 48-hour streaming window to their selected movie(s).

**Failure End Condition:** Transaction is terminated, access is denied, and user account is not charged.

**Primary Actor:** Registered Customer

**Trigger:** Registered customer selects "Rent Now" in the "Movie Rental" Portal.

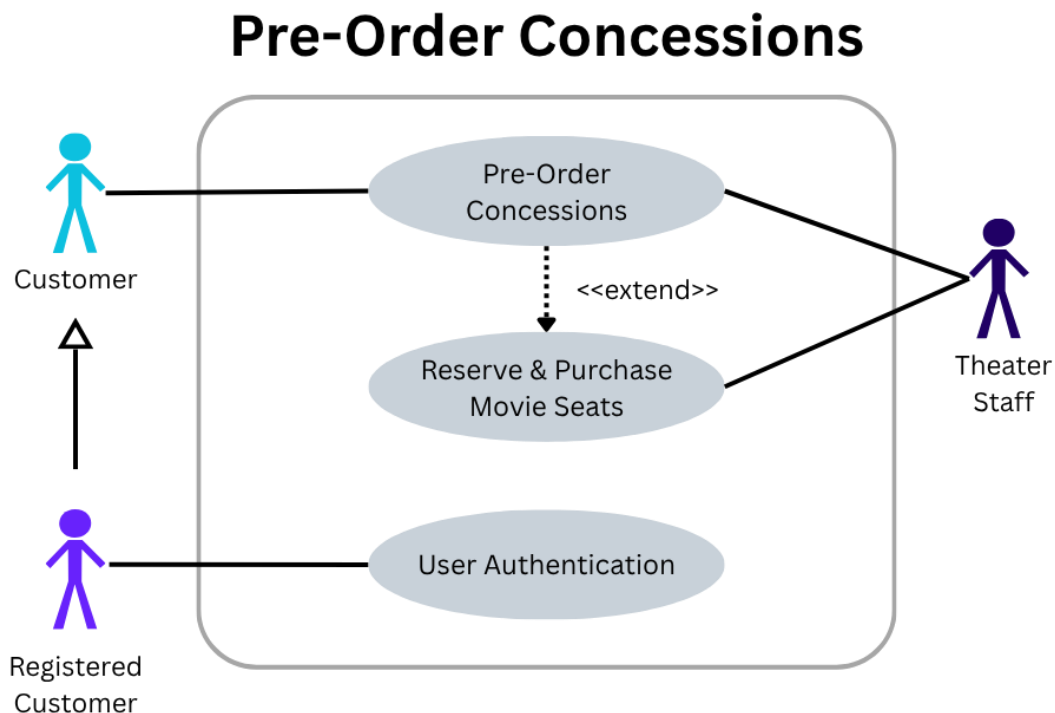
**Description:**

| Step | Action                                                                                  |
|------|-----------------------------------------------------------------------------------------|
| 1    | Customer selects a movie to rent and chooses "Rent Movie."                              |
| 2    | System displays a rental price.                                                         |
| 3    | Customer approves the rental charge and completes the checkout process.                 |
| 4    | System assigns the license to the customer's account.                                   |
| 5    | The customer may choose to play their rental, which starts the 48-hour timer.           |
| 6    | System redirects the customer to an embedded HTML5 video player with playback controls. |

**Extensions or Variations:**

| Step                              | Branching Action                                                                                                                                                      |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Incompatible Client Device</b> | If the user's device lacks the browser to support HTML5, they will not have the video decoding capability. The system will display an error if HTML5 is not detected. |

### 3.3.5 Use Case #5: Pre-Order Concessions



**Figure 6. Pre-Order Concessions Use Case Diagram**

**Goal in Context:** Allow customers to pre-order items at the concession stand.

**Preconditions:**

- (1) Concession inventory is in stock and real-time stock levels are active in the database.
- (2) Customer is completing a ticket purchase or accessing the standalone concession pre-ordering menu.

**Success End Condition:** Concession order is logged, paid for, and scheduled for pickup, then the system sends the customer a pickup notification with a QR code.

**Failure End Condition:** Concession items remain in cart, and no payment is collected.

**Primary Actor:** Customer

**Secondary Actor:** Theater Staff

**Trigger:** Customer selects the "Order Concessions" menu option during ticket checkout or in the "Concessions" portal.

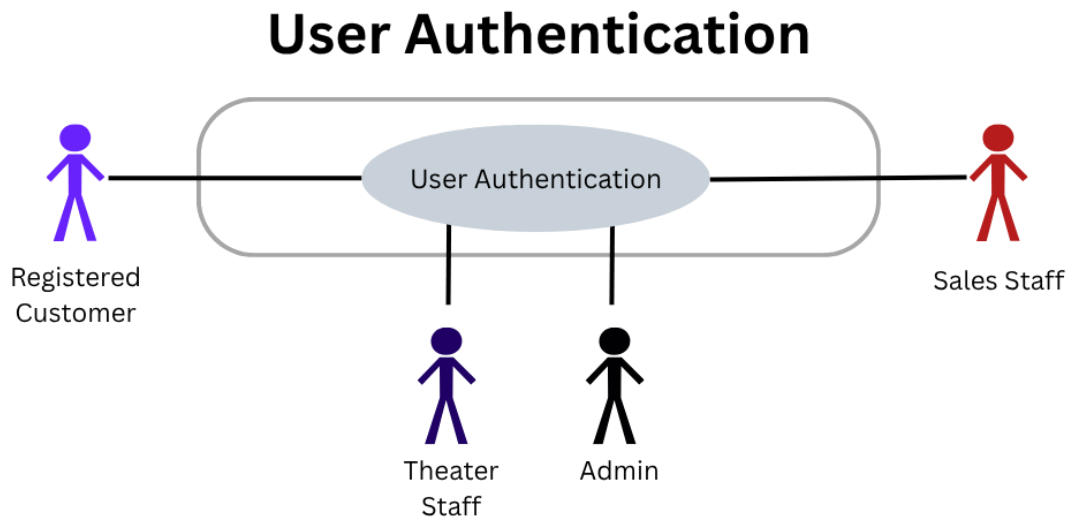
**Description:**

| Step | Action                                                                                                                                                                                                  |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | System displays a categorized menu with items and their prices.                                                                                                                                         |
| 2    | Customers select their items, quantities, and optional customizations if available.                                                                                                                     |
| 3    | System updates cart total and confirms item availability in current inventory stock.                                                                                                                    |
| 4    | Customer confirms the order and proceeds to payment. If the customer chooses to sign in, the system authenticates the customer; otherwise, checkout continues using guest information.                  |
| 5    | System processes the transaction, logs the order as "Pending," and issues the customer a receipt with a QR code.                                                                                        |
| 6    | When staff prepares the order, the system sends an automated email/SMS notification to the customer indicating that their order is ready at the concession counter and it logs the order as "Complete". |

**Extensions or Variations:**

| Step                     | Branching Action                                                                                                                                            |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Item Out of Stock</b> | If an item becomes unavailable before checkout completion, the system alerts the customer, removes the sold-out item from the order, and updates the total. |

### 3.3.6 Use Case #6: User Authentication



**Figure 7. User Authentication Use Case Diagram**

**Goal in Context:** Allows the user to log into the system.

**Preconditions:**

- (1) Customer or employee has an active account or selects a third-party identity provider like Google or Apple OAuth 2.0 Single Sign-On.
- (2) OAuth 2.0 services (Google/Apple) and cloud database auth services are online.

**Success End Condition:** User is verified with a secure session token, and access rights are granted according to the user's role (RBAC).

**Failure End Condition:** System denies access and displays an authentication error. The failed attempt will be logged.

**Primary Actor:** System User (Registered Customer/Theater Staff/ Sales Staff/Admin)

**Trigger:** User clicks "Sign in / Register" on the application interface.

**Description:**

| Step | Action                                                                                                                                        |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | The system presents login options such as email/password credentials or Third-Party Single Sign-On (Google Auth / Apple OAuth).               |
| 2    | Users select their preferred authentication method.                                                                                           |
| 3    | System verifies credentials either against securely stored password hashes or OIDC ID token provided by external providers (Google or Apple). |
| 4    | System checks user's assigned role privileges via Role-Based Access Control (RBAC).                                                           |
| 5    | System starts an encrypted user session and redirects the user to their respective portal destination.                                        |

**Extensions or Variations:**

| Step | Branching Action |
|------|------------------|
|------|------------------|

|                                         |                                                                                                                                                                              |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Invalid Credentials</b>              | System alerts the user about the incorrect password/email combination and suggests the account recovery option without revealing the account's existence for extra security. |
| <b>External SSO Service Unavailable</b> | If Google or Apple OAuth API returns a connection timeout or server error, the system alerts the user and prompts them to try the email/password login.                      |

### 3.4 Classes / Objects

| Customer                                                                                                                                                                                                                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CustomerId : UUID<br>name : String<br>email : String<br>passwordHash : String<br>age : Int<br>contactInformation : String<br>accountType : String<br>accountStatus : String                                                                                                                                                                                        |
| registerAccount() : Boolean<br>login(email: String, password: String) : Boolean<br>reserveTheater(roomId : UUID, startTime : DateTime, duration : Int) : TheaterReservation<br>purchaseTicket(showtimeId : UUID, seatIds : List<UUID>) : List<Tickets><br>rentDigitalMovie(movieId: UUID) : DigitalRental<br>orderConcessions(showtimeId : UUID) : ConcessionOrder |

#### 3.4.1 Customer

##### 3.4.1.1 Attributes

- customerId
- name
- email
- passwordHash
- age
- contactInformation
- accountType
- accountStatus

##### 3.4.1.2 Functions

- registerAccount()
- login()
- reserveTheater()
- purchaseTicket()

- rentDigitalMovie()
- orderConcessions()

Reference: 3.2.2, 3.2.3, 3.2.4, 3.2.5, 3.2.6 & Use Cases 3.3.1-3.3.6

| Movie                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| movieId : UUID<br>title : String<br>description : String<br>runtime : Int<br>showtime: Date<br>rating : String<br>availability : String<br>rentalPrice : Double<br>genre : String                              |
| getMovieInformation(movieId, description, rating, genre, title, runtime, availability) : String, UUID<br>getShowtimes(runtime, availability, showtime) : Date, String<br>getRentalOptions(rentalType) : String |

### 3.4.2 Movie

#### 3.4.2.1 Attributes

- movieId
- title
- description
- runtime
- showtime
- rating
- availability
- rentalPrice
- genre

#### 3.4.2.2 Functions

- getMovieInformation()
- getShowtimes()
- getRentalOptions()

Reference: 3.2.1, 3.2.2, 3.2.4 & Use Cases 3.3.1, 3.3.2, 3.3.4

| Employee                                                              |
|-----------------------------------------------------------------------|
| employeeId: UUID<br>name: String<br>email: String<br>username: String |

|                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| passwordHash: String<br>accountRole: String<br>rbacGroup: String<br>employmentStatus: String<br>theaterLocation: String                                                                                                                                                                                                                                       |
| manageShowtimes(movidId, showtime) : Boolean, String<br>updateConcessionInventory(itemId, quantity) : Int<br>updatePricing(itemId, newPrice) : Double, Int<br>lookupCustomerOrders(customerId) : List<Order><br>processRefund(orderId, amount) : Boolean<br>viewSalesAndAttendance(theaterLocation, dataRange) : String, Data<br>viewLogs(dateRange) : String |

### 3.4.3 Employee

#### 3.4.3.1 Attributes

- employeeId
- name
- email
- username
- passwordHash
- accountRole
- rbacGroup
- employmentStatus
- theaterLocation

#### 3.4.3.2 Functions

- manageShowtimes()
- updateConcessionInventory()
- updatePricing()
- lookupCustomerOrders()
- processRefund()
- viewSalesAndAttendance()
- viewLogs()

Reference: 3.2.5, 3.5.4, 3.2.7 & Use Case 3.3.6

### 3.4.4 MovieCatalog

#### 3.4.4.1 Attributes

- availableMovies
- currentlyStreamingMovies
- title
- movieCategories
- movieAvailability
- digitalMovies
- rentalPrices

#### 3.4.4.2 Functions

- displayDigitalMovieCatalog()
- filterMoviesByCategory()
- searchMovies()
- getMovieDetails()
- getAvailableShowtimes()
- getRentalOptionsAndPrices()

Reference: 3.1.1, 3.2.1 & Use Cases 3.3.1, 3.3.4

### 3.4.5 ConcessionOrder

#### 3.4.5.1 Attributes

- orderId
- customer
- associatedShowtime
- itemsAndQuantities
- orderTotal
- orderStatus
- qrCode

#### 3.4.5.2 Functions

- addItem()
- removeItem()
- checkInventory()
- calculateTotal()
- updateOrderStatus()
- generateQrCode()

Reference: 3.2.5 & Use Cases 3.3.5

### 3.4.6 Showtime

#### 3.4.6.1 Attributes

- showtimeId
- associatedMovie
- associatedTheaterRoom
- dateAndTime
- baseTicketPrice

#### 3.4.6.2 Functions

- getMovieTitle()
- getMovieRating()
- getDateAndTime()
- getAvailableSeats()
- getRoomId()

Reference: 3.2.1, 3.2.2 & Use Cases 3.3.1 and 3.3.2

### **3.4.7 Payment**

#### 3.4.7.1 Attributes

- paymentId
- paymentAmount
- paymentStatus
- paymentMethod
- transactionId
- dateTime
- confirmation

#### 3.4.7.2 Functions

- processPayment()
- validatePayment()
- recordPayment()
- processRefund()
- confirmPayment()

Reference: 3.2.2, 3.2.3, 3.2.4, 3.2.5, 3.5.4 & Use Cases 3.3.2, 3.3.3, 3.3.4, 3.3.5

### **3.4.8 DigitalRental**

#### 3.4.8.1 Attributes

- rentalId
- movieTitle
- rentalPrice
- rentalStatus
- rentalStartTime
- rentalExpirationTime
- digitalLicense

#### 3.4.8.2 Functions

- displayRentalInformation()
- processRental()
- checkRentalExpiration()
- provideMovieAccess()
- verifyDigitalLicense()

Reference: 3.2.4, 3.5.4, & Use Cases 3.3.4

### **3.4.9 TheaterReservation**

#### 3.4.9.1 Attributes

- reservationId
- customerId

- roomId
- reservationDate
- startTime
- duration
- guestCount
- eventType
- reservationStatus
- totalPrice

#### 3.4.9.2 Functions

- createReservation()
- calculatePrice()
- confirmReservation()
- cancelReservation()

Reference: 3.1.1, 3.2.3, & Use Cases 3.3.3

## 3.5 Non-Functional Requirements

### 3.5.1 Performance

- At least 95% of standard page loads, including the seat map, checkout, movie catalog, and concession ordering pages, shall complete within 2 seconds under normal load conditions.
- The system shall confirm/decline a payment transaction with the payment processor within 5 seconds of submission.
- The interactive seat map shall reflect a true seat status change (reservation, hold/release) to all connected clients within 3 seconds.
- The system will support at least 500 simultaneous users using the system to browse, reserve seats, or check out without response times delaying beyond set targets above.

### 3.5.2 Reliability

- Any failed/declined payment transactions shall be fully rolled back so that no seat, private theater room, or digital rental is held or granted without a corresponding successful payment recorded.
- Seat holds that expire (see 3.2.2.5) shall reliably release said seats back into the inventory without needing manual release/staff intervention.
- The system shall have an MTBF of at least 720 hours of continuous uptime.
- The system shall log all failed transactions, authentication errors, and unhandled system errors for review by admins.

### 3.5.3 Availability

- The system will be fully available 24/7 to customers, with the exception of scheduled maintenance windows, which will be announced in advance for customers to plan accordingly.
- All planned downtime shall not exceed 1 hour per calendar month and shall be scheduled during historically low-traffic hours.

- System uptime shall meet or exceed 99%, measured monthly, on the selected cloud hosting provider (AWS/GCP).

#### **3.5.4 Security**

- The system shall encrypt sensitive non-password data when it is at rest using AES-256 or a comparable standard and shall store user passwords using a secure one-way password hashing method.
- The system shall comply with PCI-DSS requirements and shall never store any raw payment methods, and all card data shall be tokenized and handled by Stripe.
- Access to admin and staff functions shall be handled by RBAC, to restrict functionality according to each employee's role (See Section 3.2.7.3).
- Authentication shall support OAuth 2.0/OIDC for third-party SSO in addition to a securely hashed email/password login.
- All failed attempts at logging in shall be rate-limited and logged to help detect and prevent brute-force attacks.

#### **3.5.5 Maintainability**

- The system shall be developed in a modular architecture that separates the front-end, RESTful API, and PostgreSQL database layers, so each part can be updated alone without affecting the others.
- All back-end API endpoints shall be written in detail for documentation to support future development, debugging, and onboarding of new members to the team.
- The codebase shall follow a consistent documentation standard for coding and shall be maintained under version control (e.g., Git) with a defined branching/review process.

#### **3.5.6 Portability**

- The system shall be fully functional on the latest two major versions of all popular web browsers (Chrome, Firefox, Safari, and Edge), consistent with the constraints put forth in Section 2.4.
- The system's responsive design shall support desktop, tablet, and phone screen resolutions without loss of core functionality, consistent with the hardware interfaces highlighted in Section 3.1.2.
- The system shall not depend on any operating system-specific technologies, allowing deployment to either AWS or GCP (see Section 2.4) without requiring code changes.

### **3.6 Inverse Requirements**

*State any \*useful\* inverse requirements.*

### **3.7 Design Constraints**

*Specify design constraints imposed by other standards, company policies, hardware limitation, etc. that will impact this software project.*

### **3.8 Logical Database Requirements**

*Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc.*

### 3.9 Other Requirements

*Catchall section for any additional requirements.*

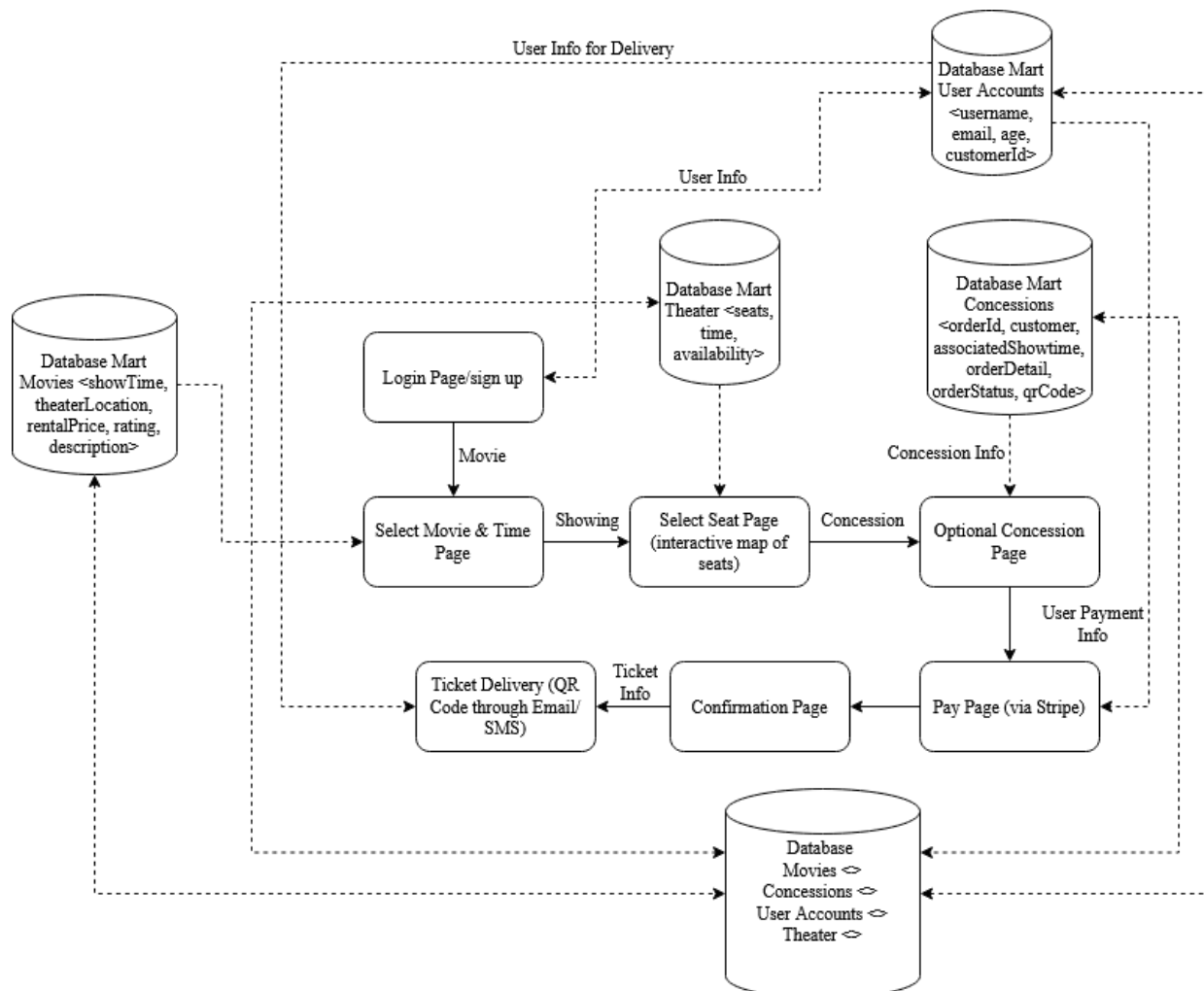
## 4. Software Design Specification

The following sections explain each class shown in the Seven Fold Cinema UML Class Diagram. Each description covers the class's purpose, the information it stores, its main operations, and its relationships with other classes in the system. Together, the class diagram and class descriptions outline the logical structure developers can use when building the system.

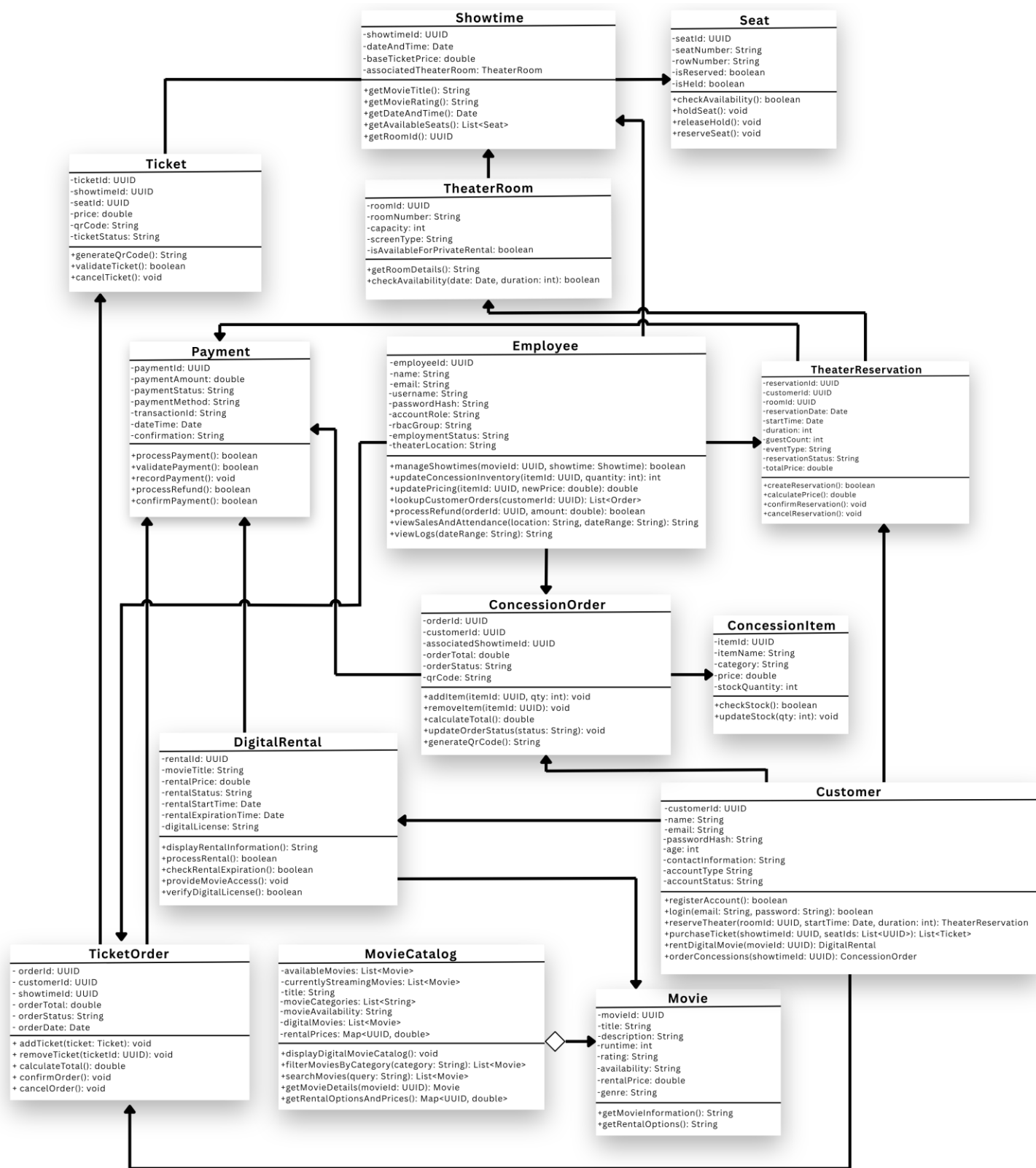
### 4.1 System Description

Seven Fold Cinema is a web-based movie theater management and ticketing platform designed for the entertainment of its users. The system integrates user authentication, movie renting, real-time movie scheduling, interactive seat selection, optional concession pre-ordering, and secure payment processing all into one single web-application.

### 4.2 Software Architecture Overview



## 4.3 UML Class Design



## 4.4 UML Class Descriptions

### 4.4.1 Customer

The Customer class represents a user who uses the Seven Fold Cinema website to buy tickets, reserve private theater rooms, rent digital movies, and order concessions. It stores details like customer ID, name, email, password hash, contact info, account type, and account status. The class allows users to register, log in, and perform the main activities available to customers. Each customer can be connected to multiple TicketOrder, TheaterReservation, DigitalRental, and ConcessionOrder objects at any time.

| Attribute          | Datatype | Description                                                                                                                                                                                                                    |
|--------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| customerId         | UUID     | Unique identifier assigned to each customer account. It allows the system to distinguish one customer from all other customers and associate purchases, reservations, rentals, and concession orders with the correct account. |
| name               | String   | Stores the customer's name for account identification and customer records.                                                                                                                                                    |
| email              | String   | Stores the customer's email address. It may be used as an account identifier and for communication such as purchase or reservation confirmations.                                                                              |
| passwordHash       | String   | Stores the secure one-way hash of the customer's password. Plaintext passwords are not stored.                                                                                                                                 |
| age                | int      | Stores the customer's age. This may be used where age-related restrictions or account requirements apply.                                                                                                                      |
| contactInformation | String   | Stores additional customer contact information, such as a telephone number or preferred contact details.                                                                                                                       |
| accountType        | String   | Identifies the type or category of customer account used by the system.                                                                                                                                                        |
| accountStatus      | String   | Represents the current condition of the account, such as active, suspended, or disabled.                                                                                                                                       |

| Operation                                                                        | Parameters/Return                                                                        | Description                                                                                                                                                          |
|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| registerAccount(): boolean                                                       | No parameters; returns boolean                                                           | Creates a new customer account using information supplied during registration. Returns true when registration succeeds and false when the account cannot be created. |
| login(email: String, password: String): boolean                                  | email: identifies the account; password: contains the supplied password; returns boolean | Authenticates a customer by locating the account associated with the supplied email and verifying the supplied password against the stored password hash.            |
| reserveTheater(roomId: UUID, startTime: Date, duration: int): TheaterReservation | Room identifier, starting time, and rental duration; returns TheaterReservation          | Initiates a private theater reservation for the specified room and time. The returned object represents the reservation being created.                               |

|                                                                     |                                                                                         |                                                                                                                                           |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| purchaseTicket(showtimeId: UUID, seatIds: List<UUID>): List<Ticket> | Showtime identifier and a collection of selected seat identifiers; returns List<Ticket> | Initiates the purchase of one or more tickets for a selected showtime and seats. Successful processing produces a list of ticket objects. |
| rentDigitalMovie(movieId: UUID): DigitalRental                      | Identifier of the digital movie; returns DigitalRental                                  | Initiates a digital movie rental and creates a rental object associated with the customer's account.                                      |
| orderConcessions(showtimeId: UUID): ConcessionOrder                 | Identifier of the related showtime; returns ConcessionOrder                             | Creates a concession order associated with the customer's theater visit and returns the new order for item selection and checkout.        |

#### 4.4.2 Employee

The Employee class represents theater staff who handle administrative or staff tasks in the Seven Fold Cinema system. It stores information like employee ID, name, email, username, password hash, role, RBAC group, employment status, and assigned theater location. Employees can manage showtimes, update concession inventory and prices, find customer orders, assist with private theater reservations, process refunds, and view operational information. Access to these features depends on the employee's role.

| Attribute        | Datatype | Description                                                                                                    |
|------------------|----------|----------------------------------------------------------------------------------------------------------------|
| employeeId       | UUID     | Unique identifier assigned to an employee record.                                                              |
| name             | String   | Stores the employee's name.                                                                                    |
| email            | String   | Stores the employee's email address for identification or communication.                                       |
| username         | String   | Stores the username used when accessing employee functions.                                                    |
| passwordHash     | String   | Stores the one-way hash used to verify the employee's password securely.                                       |
| accountRole      | String   | Identifies the employee's role, such as theater staff, sales staff, or administrator.                          |
| rbacGroup        | String   | Identifies the Role-Based Access Control group that determines which system functions the employee may access. |
| employmentStatus | String   | Describes whether the employee account is currently active, inactive, suspended, or otherwise restricted.      |
| theaterLocation  | String   | Identifies the theater location with which the employee is associated.                                         |

| Operation                                                   | Parameters/Return                                     | Description                                                                                                                                          |
|-------------------------------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| manageShowtimes(movieId: UUID, showtime: Showtime): boolean | Movie identifier and Showtime object; returns boolean | Allows an authorized employee to create, modify, or otherwise manage scheduling information. Returns whether the requested operation was successful. |
| updateConcessionInventory(itemId: UUID, quantity: int): int | Concession item identifier and quantity; returns int  | Modifies the stored inventory quantity for a concession item and returns the resulting inventory quantity.                                           |

|                                                                     |                                                       |                                                                                                                                                        |
|---------------------------------------------------------------------|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| updatePricing(itemId: UUID, newPrice: double): double               | Item identifier and new price; returns double         | Changes the price of an item and returns the current price stored by the system.                                                                       |
| lookupCustomerOrders(customerId: UUID): List<Order>                 | Customer identifier; returns List<Order>              | Retrieves orders associated with the specified customer so authorized staff can assist with purchases, reservations, or other customer-service issues. |
| processRefund(orderId: UUID, amount: double): boolean               | Order identifier and refund amount; returns boolean   | Initiates a refund for a customer transaction. Returns whether the refund was successfully processed.                                                  |
| viewSalesAndAttendance(location: String, dateRange: String): String | Theater location and reporting period; returns String | Produces sales and attendance information for the selected theater and date range.                                                                     |
| viewLogs(dateRange: String): String                                 | Reporting period; returns String                      | Retrieves system or administrative activity logs for the specified date range for accountability and troubleshooting.                                  |

#### 4.4.3 Ticket

The Ticket class represents access to a specific theatrical performance. It includes a unique ID, purchase price, QR code, and status. Each ticket is linked to a particular Showtime and is created as part of a TicketOrder. The class can generate a QR code for admission, validate the ticket, and cancel the ticket if required.

| Attribute    | Datatype | Description                                                                        |
|--------------|----------|------------------------------------------------------------------------------------|
| ticketId     | UUID     | Unique identifier assigned to the ticket.                                          |
| showtimeId   | UUID     | Identifies the showtime for which the ticket is valid.                             |
| seatId       | UUID     | Identifies the seat reserved by this ticket.                                       |
| price        | double   | Stores the price paid for the ticket.                                              |
| qrCode       | String   | Stores the generated QR-code value used to identify and validate the ticket.       |
| ticketStatus | String   | Represents the current condition of the ticket, such as active, used, or canceled. |

| Operation                 | Parameters/Return              | Description                                                                                                           |
|---------------------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| generateQrCode(): String  | No parameters; returns String  | Generates the QR-code identifier associated with the ticket and returns the resulting encoded value.                  |
| validateTicket(): boolean | No parameters; returns boolean | Determines whether the ticket is valid for admission based on its status and associated showing.                      |
| cancelTicket(): void      | No parameters; returns nothing | Changes the ticket so that it is no longer valid for admission, normally as part of a cancellation or refund process. |

#### 4.4.4 TicketOrder

The TicketOrder class manages a customer's purchase of one or more theater tickets. It groups individual Ticket objects into a single transaction and records the order ID, total price, order status, and transaction date. The class lets users add or remove tickets before checkout, calculate the total cost, confirm the order after payment, and cancel the order if needed. It connects the customer's ticket selection to the related Payment.

| Attribute   | Datatype | Description                                                                        |
|-------------|----------|------------------------------------------------------------------------------------|
| orderId     | UUID     | Unique identifier for the ticket order.                                            |
| customerId  | UUID     | Identifies the customer who placed the order.                                      |
| showtimeId  | UUID     | Identifies the theatrical showtime associated with the order.                      |
| orderTotal  | double   | Stores the combined monetary cost of all tickets included in the order.            |
| orderStatus | String   | Describes the current state of the order, such as pending, confirmed, or canceled. |
| orderDate   | Date     | Stores the date and time at which the order was created or placed.                 |

| Operation                          | Parameters/Return                                   | Description                                                                                                     |
|------------------------------------|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| addTicket(ticket: Ticket): void    | A Ticket object; returns nothing                    | Adds a selected ticket to the ticket order.                                                                     |
| removeTicket(ticketId: UUID): void | Identifier of the ticket to remove; returns nothing | Removes a ticket from the order before the transaction is finalized.                                            |
| calculateTotal(): double           | No parameters; returns double                       | Calculates and returns the combined price of all tickets contained in the order.                                |
| confirmOrder(): void               | No parameters; returns nothing                      | Marks the ticket order as "confirmed" after the required conditions, such as a successful payment has been met. |
| cancelOrder(): void                | No parameters; returns nothing                      | Cancels the ticket order and allows associated resources, such as held seats, to be released when appropriate.  |

#### 4.4.5 Showtime

The Showtime class represents a scheduled theatrical performance in a physical room at Seven Fold Cinema. Unlike the Movie class, which is for digital rentals, Showtime is for the physical theater schedule. It stores the showtime ID, scheduled date and time, ticket price, and other details needed to identify the film. The class can provide showtime information, locate the assigned theater room, and determine which seats are available.

| Attribute       | Datatype | Description                                                                                       |
|-----------------|----------|---------------------------------------------------------------------------------------------------|
| showtimeId      | UUID     | Unique identifier assigned to the scheduled showing.                                              |
| dateAndTime     | Date     | Stores the scheduled date and starting time of the showing.                                       |
| baseTicketPrice | double   | Stores the standard ticket price for the showing before any additional pricing rules are applied. |

|                       |             |                                                                   |
|-----------------------|-------------|-------------------------------------------------------------------|
| associatedTheaterRoom | TheaterRoom | References the physical theater room in which the showing occurs. |
|-----------------------|-------------|-------------------------------------------------------------------|

| Operation                       | Parameters/Return                 | Description                                                                           |
|---------------------------------|-----------------------------------|---------------------------------------------------------------------------------------|
| getMovieTitle(): String         | No parameters; returns String     | Returns the title of the film scheduled for the showtime.                             |
| getMovieRating(): String        | No parameters; returns String     | Returns the content rating associated with the showtime.                              |
| getDateAndTime(): Date          | No parameters; returns Date       | Returns the scheduled date and starting time of the showing.                          |
| getAvailableSeats(): List<Seat> | No parameters; returns List<Seat> | Retrieves the collection of seats currently available for selection for this showing. |
| getRoomId(): UUID               | No parameters; returns UUID       | Returns the unique identifier of the theater room assigned to the showtime.           |

#### 4.4.6 Seat

The Seat class represents a single seat available for a specific theatrical performance. It includes the seat ID, row, seat number, and current status. A seat can be available, temporarily held, or reserved. The class lets the system check seat availability, place a temporary hold during checkout, reserve the seat after purchase, and release the seat if the hold expires or the transaction is canceled.

| Attribute  | Datatype | Description                                                                                                                |
|------------|----------|----------------------------------------------------------------------------------------------------------------------------|
| seatId     | UUID     | Unique identifier assigned to the seat.                                                                                    |
| seatNumber | String   | Identifies the seat number within its theater row.                                                                         |
| rowNumber  | String   | Identifies the row in which the seat is located.                                                                           |
| isReserved | boolean  | Indicates whether the seat has already been reserved through a successful ticket purchase.                                 |
| isHeld     | boolean  | Indicates whether the seat is temporarily unavailable because it is being held during another customer's checkout process. |

| Operation                    | Parameters/Return              | Description                                                                                        |
|------------------------------|--------------------------------|----------------------------------------------------------------------------------------------------|
| checkAvailability(): boolean | No parameters; returns boolean | Checks isReserved and isHeld to determine whether the seat can currently be selected.              |
| holdSeat(): void             | No parameters; no returns      | Temporarily places the seat on hold while the customer completes checkout.                         |
| releaseHold(): void          | No parameters; no returns      | Removes the temporary hold so the seat becomes available again if checkout expires or is canceled. |
| reserveSeat(): void          | No parameters; no returns      | Marks the seat as reserved after a successful ticket purchase.                                     |

#### 4.4.7 TheaterRoom

The TheaterRoom class represents a real auditorium in Seven Fold Cinema. It stores the room ID, name or number, seating capacity, room type, rental price, and status. Theater rooms are used

for both scheduled Showtimes and private reservations. The class allows the system to check room availability, get capacity details, and provide the current rental price.

| Attribute                   | Datatype | Description                                                                                      |
|-----------------------------|----------|--------------------------------------------------------------------------------------------------|
| roomId                      | UUID     | Unique identifier for the theater room.                                                          |
| roomNumber                  | String   | Room or auditorium number.                                                                       |
| capacity                    | int      | Max number of guests the room can accommodate.                                                   |
| screenType                  | String   | Describes the screen configuration.                                                              |
| isAvailableForPrivateRental | boolean  | Indicates whether the room is eligible to be offered through the private theater rental service. |

| Operation                                             | Parameters/Return                                             | Description                                                                                                                   |
|-------------------------------------------------------|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| getRoomDetails(): String                              | No parameters; returns String                                 | Returns descriptive information about the theater room, such as its room number, capacity, and screen type.                   |
| checkAvailability(date: Date, duration: int): boolean | Requested date/time and reservation duration; returns boolean | Determines whether the room is available for the requested period by considering existing schedules and private reservations. |

#### 4.4.8 TheaterReservation

The TheaterReservation class handles a customer's booking of an entire theater room for private use. It includes the reservation ID, requested date and start time, duration, number of guests, event type, reservation status, and total price. Each reservation is linked to a Customer and a TheaterRoom, and employees can help manage the reservation. The class allows creating reservations, calculating rental prices, confirming bookings after payment, and canceling reservations when needed.

| Attribute         | Datatype | Description                                                                                               |
|-------------------|----------|-----------------------------------------------------------------------------------------------------------|
| reservationId     | UUID     | Unique identifier assigned to the private reservation.                                                    |
| customerId        | UUID     | Identifies the registered customer responsible for the reservation.                                       |
| roomId            | UUID     | Identifies the theater room being reserved.                                                               |
| reservationDate   | Date     | Stores the date of the private event.                                                                     |
| startTime         | Date     | Stores the requested starting time of the reservation.                                                    |
| duration          | int      | Stores the requested length of the reservation, normally expressed in minutes or another documented unit. |
| guestCount        | int      | Stores the number of guests expected to attend.                                                           |
| eventType         | String   | Describes the intended use of the room, such as a private screening or event.                             |
| reservationStatus | String   | Indicates whether the reservation is pending, confirmed, canceled, or in another supported state.         |
| totalPrice        | double   | Stores the calculated cost of the reservation.                                                            |

| Operation | Parameters/Return | Description |
|-----------|-------------------|-------------|
|-----------|-------------------|-------------|

|                                 |                                   |                                                                                                                            |
|---------------------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| createReservation():<br>boolean | No parameters; returns<br>boolean | Attempts to create the reservation after the required customer, room, scheduling, and event information has been provided. |
| calculatePrice():<br>double     | No parameters; returns<br>double  | Calculates and returns the price of the reservation according to the selected room, time, and duration.                    |
| confirmReservation():<br>void   | No parameters; returns<br>nothing | Changes the reservation to a confirmed state after the system verifies the required conditions, including payment.         |
| cancelReservation():<br>void    | No parameters; returns<br>nothing | Cancels the private theater reservation and makes the corresponding time slot available again when appropriate.            |

#### 4.4.9 ConcessionOrder

The ConcessionOrder class manages a customer's order for food and drinks. It stores the order ID, total price, order status, and a QR or pickup code. Each order includes one or more ConcessionItem selections and can be placed alone or with a theater visit. The class lets users add or remove items, check inventory, calculate the total cost, update the order status, and generate a QR code for pickup.

| Attribute            | Datatype | Description                                                                                                          |
|----------------------|----------|----------------------------------------------------------------------------------------------------------------------|
| orderId              | UUID     | Unique identifier assigned to the concession order.                                                                  |
| customerId           | UUID     | Identifies the customer who placed the order.                                                                        |
| associatedShowtimeId | UUID     | Identifies the showing associated with the concession order when it is linked to a ticket purchase or theater visit. |
| orderTotal           | double   | Stores the total cost of all concession items in the order.                                                          |
| orderStatus          | String   | Identifies the current processing state of the order, such as pending, ready, completed, or canceled.                |

| Operation                                  | Parameters/Return                                             | Description                                                                                             |
|--------------------------------------------|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| addItem(itemId: UUID, qty: int):<br>void   | Item identifier and<br>requested quantity;<br>returns nothing | Adds the requested quantity of the specified concession item to the order.                              |
| removeItem(itemId: UUID): void             | Item identifier; returns<br>nothing                           | Removes the specified concession item from the order.                                                   |
| calculateTotal(): double                   | No parameters;<br>returns double                              | Calculates the combined cost of all concession items and returns the order total.                       |
| updateOrderStatus(status: String):<br>void | New status value;<br>returns nothing                          | Changes the order's current processing status as the concession staff prepares and completes the order. |
| generateQrCode(): String                   | No parameters;<br>returns String                              | Generates and returns the QR-code identifier associated with the concession order.                      |

#### 4.4.10 ConcessionItem

The ConcessionItem class represents a single food, drink, or other concession item sold at Seven Fold Cinema. It includes the item ID, name, category, price, current stock quantity, and any

supported customization options. The class lets the system check if the requested quantity is available, update inventory levels, and change the item's price. ConcessionItem objects are selected and added to ConcessionOrder objects.

| Attribute     | Datatype | Description                                                           |
|---------------|----------|-----------------------------------------------------------------------|
| itemId        | UUID     | Unique identifier assigned to the concession product.                 |
| itemName      | String   | Name of the concession item.                                          |
| category      | String   | Category used to organize the item, such as food, candy, or beverage. |
| price         | double   | Current price of one unit of the item.                                |
| stockQuantity | int      | Current number of units available for ordering.                       |

| Operation                   | Parameters/Return                                             | Description                                                                                          |
|-----------------------------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| checkStock(): boolean       | No parameters; returns boolean                                | Determines whether the concession item currently has inventory available for purchase.               |
| updateStock(qty: int): void | Quantity by which inventory is to be updated; returns nothing | Modifies the stored inventory quantity after purchases, restocking, or authorized staff adjustments. |

#### 4.4.11 Payment

The Payment class represents a financial transaction carried out in the Seven Fold Cinema system and includes the payment ID, amount, status, method, transaction ID, date and time, and confirmation details. Payments can be linked to a TicketOrder, ConcessionOrder, TheaterReservation, or DigitalRental. The class allows submitting payments, verifying results, recording transactions, confirming successful payments, and processing refunds. The system uses the external Stripe payment service to handle card details instead of storing them in our databases, which provides an extra layer of security for our customers.

| Attribute     | Datatype | Description                                                                           |
|---------------|----------|---------------------------------------------------------------------------------------|
| paymentId     | UUID     | Unique identifier assigned to the payment record.                                     |
| paymentAmount | double   | Amount paid.                                                                          |
| paymentStatus | String   | Current transaction state, such as pending, approved, declined, or refunded.          |
| paymentMethod | String   | Identifies the payment method or supported payment category used for the transaction. |
| transactionId | String   | Stores the transaction identifier returned by the external payment processor.         |
| dateTime      | Date     | Stores when the payment transaction occurred.                                         |
| confirmation  | String   | Stores payment confirmation information returned after successful processing.         |

| Operation                 | Parameters/Return              | Description                                                                                                                          |
|---------------------------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| processPayment(): boolean | No parameters; returns boolean | Submits the prepared payment transaction to the approved payment processor like Stripe API and reports whether processing succeeded. |

|                               |                                   |                                                                                                                                                     |
|-------------------------------|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| validatePayment():<br>boolean | No parameters;<br>returns boolean | Validates the transaction result and determines whether the payment can be accepted by the Seven Fold Cinema system.                                |
| recordPayment():<br>void      | No parameters;<br>returns nothing | Saves the non-sensitive payment transaction information and result for order history and administrative records. No credit card details are stored. |
| processRefund():<br>boolean   | No parameters;<br>returns boolean | Attempts to return an eligible payment through the approved payment processor and indicates whether the refund succeeded.                           |
| confirmPayment():<br>boolean  | No parameters;<br>returns boolean | Determines whether the transaction has reached a successfully confirmed state so the associated purchase, reservation, or rental can be finalized.  |

#### 4.4.12 DigitalRental

The DigitalRental class includes the rental ID, rental price, rental status, rental start time, expiration time, and digital license information. Each rental is linked to a specific Customer and Movie, and the class's operations allow processing the rental, initiating playback, checking whether the rental period has expired, verifying the digital license, and granting access to the movie as long as the rental is valid.

| Attribute            | Datatype | Description                                                                                                            |
|----------------------|----------|------------------------------------------------------------------------------------------------------------------------|
| rentalId             | UUID     | Unique identifier assigned to the rental transaction.                                                                  |
| movieTitle           | String   | Stores the title of the digital movie being rented.                                                                    |
| rentalPrice          | double   | Amount charged for the digital rental.                                                                                 |
| rentalStatus         | String   | Describes whether the rental is pending, active, expired, or otherwise unavailable.                                    |
| rentalStartTime      | Date     | Stores when the rental viewing period begins, based on the customer's start of playback.                               |
| rentalExpirationTime | Date     | Stores when access to the rental expires. This represents the end of the 48-hour viewing period after playback begins. |
| digitalLicense       | String   | Stores or references the authorization needed to verify that the customer has legal access to the rented movie.        |

| Operation                             | Parameters/Return                 | Description                                                                                                                              |
|---------------------------------------|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| displayRentalInformation():<br>String | No parameters; returns<br>String  | Returns rental information such as title, price, status, and remaining viewing period.                                                   |
| processRental(): boolean              | No parameters; returns<br>boolean | Processes the rental request and determines whether the rental can be activated after required conditions such as payment are satisfied. |
| checkRentalExpiration():<br>boolean   | No parameters; returns<br>boolean | Compares the current time with the stored expiration time to determine whether the rental remains valid.                                 |
| provideMovieAccess(): void            | No parameters; returns<br>nothing | Provides or enables playback access when the rental and digital license are valid.                                                       |
| verifyDigitalLicense():<br>boolean    | No parameters; returns<br>boolean | Verifies the digital license associated with the rental before access to the movie is permitted.                                         |

#### 4.4.13 Movie

The Movie class represents a film available in the Seven Fold Cinema digital rental catalog. It is separate from the Showtime class, which handles films shown in a physical theater. This class stores details like the movie's ID, title, description, running time, rating, availability, rental price, and genre. Its functions let the system get information about movies and offer rental options for each title. Since different customers can rent the same movie, one movie can be linked to several DigitalRental records.

| Attribute    | Datatype | Description                                          |
|--------------|----------|------------------------------------------------------|
| movieId      | UUID     | Unique identifier assigned to the movie.             |
| title        | String   | Title displayed to customers in the digital catalog. |
| description  | String   | Summary or descriptive information about the movie.  |
| runtime      | int      | Length of the movie, normally expressed in minutes.  |
| rating       | String   | Stores the movie's content rating.                   |
| availability | String   | Indicates whether the movie is currently available.  |
| rentalPrice  | double   | Price charged to rent the movie digitally.           |
| genre        | String   | Classification of the movie by genre.                |

| Operation                     | Parameters/Return             | Description                                                                         |
|-------------------------------|-------------------------------|-------------------------------------------------------------------------------------|
| getMovieInformation(): String | No parameters; returns String | Returns descriptive information associated with the movie for display to customers. |
| getRentalOptions(): String    | No parameters; returns String | Returns information describing the digital rental options available for the movie.  |

#### 4.4.14 MovieCatalog

The MovieCatalog class shows the range of movies available through the Seven Fold Cinema digital rental service. It keeps track of groups of digital movies and the categories that customers can browse or search. The class lets the system display the digital catalog, filter movies by category, search for movies by title or other criteria, get detailed information about movies, and show rental prices and options. MovieCatalog can bring multiple Movie objects together.

| Attribute                | Datatype     | Description                                                                                        |
|--------------------------|--------------|----------------------------------------------------------------------------------------------------|
| availableMovies          | List<Movie>  | Collection of movies currently available through the catalog.                                      |
| currentlyStreamingMovies | List<Movie>  | Collection of movies currently available for digital streaming or rental.                          |
| title                    | String       | Stores a title or identifying label associated with the catalog.                                   |
| movieCategories          | List<String> | Collection of categories used to organize and filter the digital movie catalog.                    |
| movieAvailability        | String       | Stores information representing the availability state used by the catalog.                        |
| digitalMovies            | List<Movie>  | Collection of movies offered specifically through the digital rental portion of Seven Fold Cinema. |

|              |                   |                                                                           |
|--------------|-------------------|---------------------------------------------------------------------------|
| rentalPrices | Map<UUID, double> | Associates each movie's UUID with its corresponding digital rental price. |
|--------------|-------------------|---------------------------------------------------------------------------|

| Operation                                             | Parameters/Return                       | Description                                                                        |
|-------------------------------------------------------|-----------------------------------------|------------------------------------------------------------------------------------|
| displayDigitalMovieCatalog():<br>void                 | No parameters; returns nothing          | Produces or displays the collection of digital movies that customers may browse.   |
| filterMoviesByCategory(category: String): List<Movie> | Category text; returns List<Movie>      | Filters the catalog and returns only movies belonging to the specified category.   |
| searchMovies(query: String): List<Movie>              | Search text; returns List<Movie>        | Searches catalog information using the supplied query and returns matching movies. |
| getMovieDetails(movieId: UUID): Movie                 | Movie identifier; returns Movie         | Retrieves the complete movie object associated with the specified identifier.      |
| getRentalOptionsAndPrices(): Map<UUID, double>        | No parameters; returns Map<UUID,double> | Returns the relationship between available digital movies and their rental prices. |

## 4.5 Development Plan and Timeline

### Task partitioning

The system divides naturally into the following work packages, grouped by the classes that interact most closely in the diagram:

1. Ticketing & Seating  
Classes: Ticket, Seat, TicketOrder  
Covers seat selection/hold logic, ticket generation, QR codes, and order management for ticket purchases.
2. Showtime & Catalog  
Classes: Showtime, TheaterRoom, Movie, MovieCatalog  
Covers movie listings, showtime scheduling, room availability, and search/filter functionality.
3. Customer, Account Management, Employee, & Admin Tools  
Classes: Customer, Employee  
Cover registration, login/authentication, and the customer-facing methods that kick off purchases, rentals, and reservation. It also covers showtime management, inventory/pricing updates, refunds, and reporting (sales, attendance, logs).
4. Payment, Concessions, Digital Rentals & Theater Reservation (Private Events)  
Classes: Payment, ConcessionOrder, ConcessionItem, DigitalRental, TheaterReservation

Covers payment processing/validation, concession ordering, and inventory/stock tracking. It also covers rental purchase flow, license verification, and expiration handling. It also covers private room booking, pricing calculation, and confirmation/cancellation.

### Team Member Responsibilities:

| Team Member | Work Package(s)                                        | Key Deliverables                                                                            |
|-------------|--------------------------------------------------------|---------------------------------------------------------------------------------------------|
|             | Ticketing, Seating, Showtime & Catalog                 | Ticket, Seat, TicketOrder classes, Showtime, TheaterRoom, Movie, MovieCatalog classes       |
|             | Customer, Account Management, Employee, & Admin Tools  | Customer class, auth flow, Employee class, reporting features                               |
|             | Payments, concessions, Digital Rentals, & Reservations | Payment, ConcessionOrder, ConcessionItem classes, DigitalRental, TheaterReservation classes |

### Timeline:

| Phase                                | Weeks               | Tasks                                                                                   |
|--------------------------------------|---------------------|-----------------------------------------------------------------------------------------|
| Phase 1: Setup & design finalization | Week 1              | Finalize class diagram, assign work packages, set up repo/dev environment               |
| Phase 2: Core Data Models            | Weeks 2             | Implement all classes' attributes and constructors                                      |
| Phase 3: Core Logic Implementation   | Weeks 3             | Implement business logic per work package (ticketing, payments, catalog, etc.)          |
| Phase 4: Integration                 | Week 4 (First half) | Connect subsystems (e.g., Customer -> TicketOrder -> Payment; Showtime -> MovieCatalog) |
| Phase 5: Testing & Debugging         | Week 4 (Last half)  | Unit tests per class, integration tests across workflows                                |
| Phase 6: Documentation & Report      | Week 5              | Finalize SRS sections, prepare polished final project report                            |

## **5. Analysis Models**

*List all analysis models used in developing specific requirements previously given in this SRS. Each model should include an introduction and a narrative description. Furthermore, each model should be traceable the SRS's requirements.*

### **5.1 Sequence Diagrams**

### **5.3 Data Flow Diagrams (DFD)**

### **5.2 State-Transition Diagrams (STD)**

## **6. Change Management Process**

*Identify and describe the process that will be used to update the SRS, as needed, when project scope or requirements change. Who can submit changes and by what means, and how will these changes be approved.*

## **A. Appendices**

*Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.*

*Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.*

### **A.1 Appendix 1**

### **A.2 Appendix 2**